



CS 610

GPU Compute Optimizations

Jieyang Chen

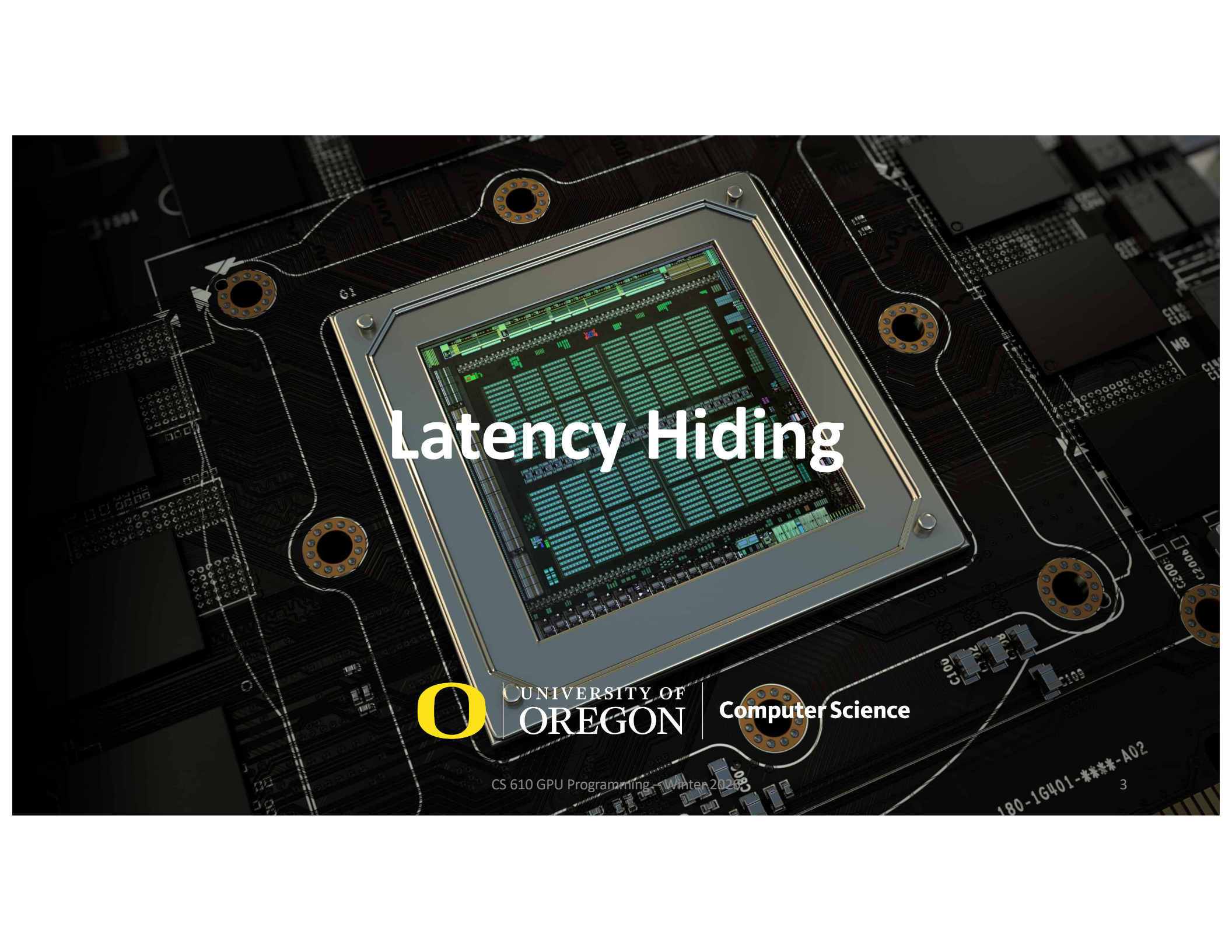


UNIVERSITY OF
OREGON

Computer Science

CS 610 GPU Programming, Winter 2026

Updates



Latency Hiding



UNIVERSITY OF
OREGON

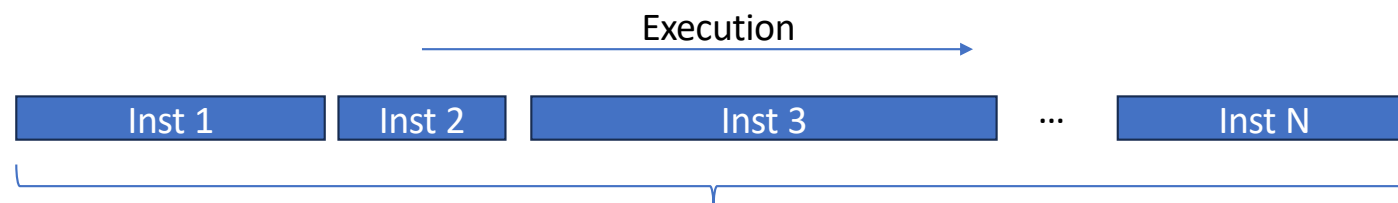
Computer Science

CS 610 GPU Programming, Winter 2026

What determines the execution time?

- For serial program, the total execution time is the sum of all instructions' execution time

*Total execution time = execution time of instruction 1 +
execution time of instruction 2 +
execution time of instruction 3 +
....
execution time of instruction N;*



Total execution time

CS 610 GPU Programming – Winter 2026

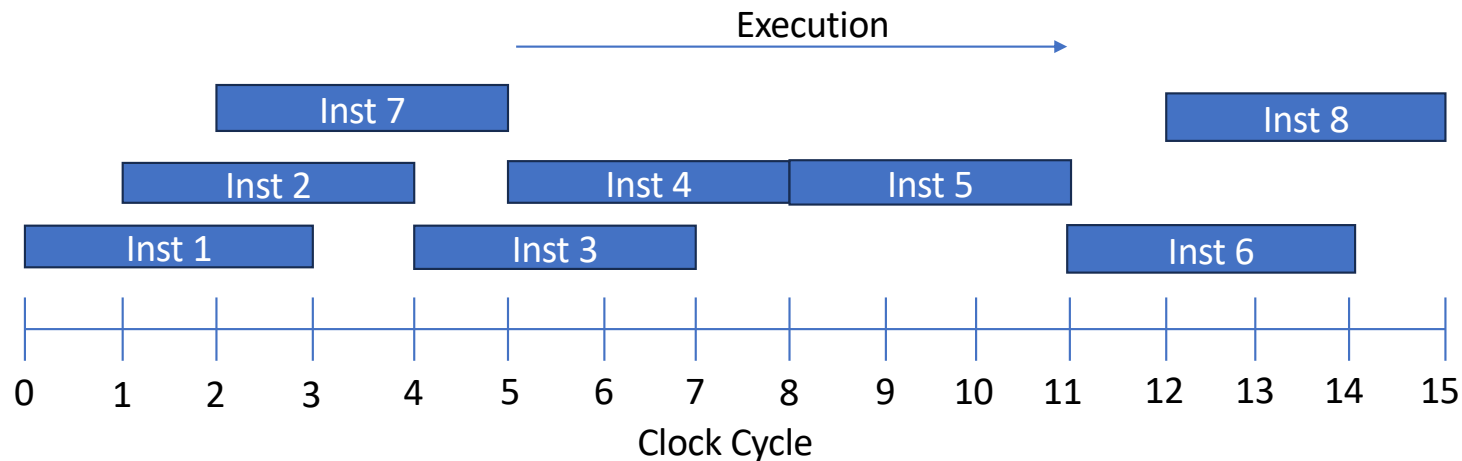


UNIVERSITY OF
OREGON

Computer Science

What determines the execution time?

- For parallel program, the total execution time is NOT the sum of all instructions' execution time since they may or may not overlap



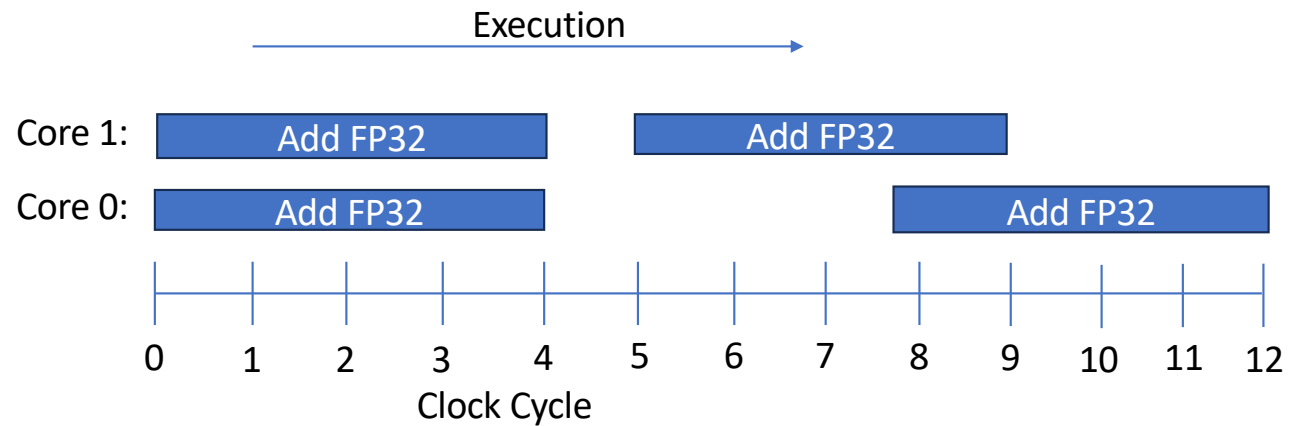
It is hard to estimate the execution time based on instructions' execution time

Why do instructions overlap?

- Parallelism on multiple cores
- Pipeline execution on the same core

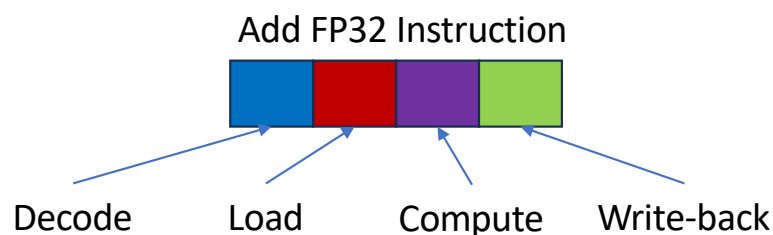
Why do instructions overlap?

- Parallelism on multiple cores



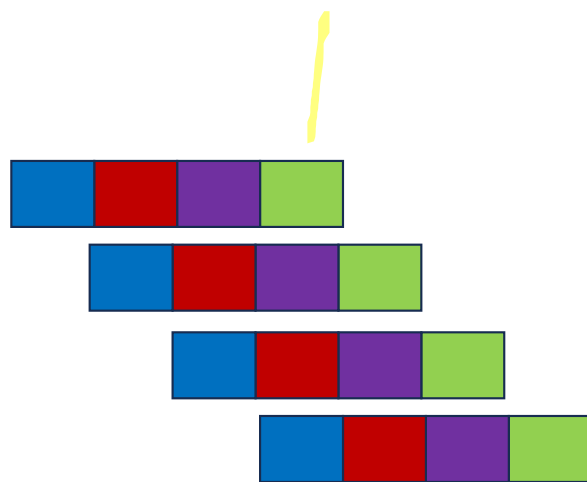
Why do instructions overlap?

- Pipeline execution on the same core
 - Executing one instruction involves multiple stages on different HW components
 - Multiple executions can be executed at the same time they are pipelined at different stages



Why do instructions overlap?

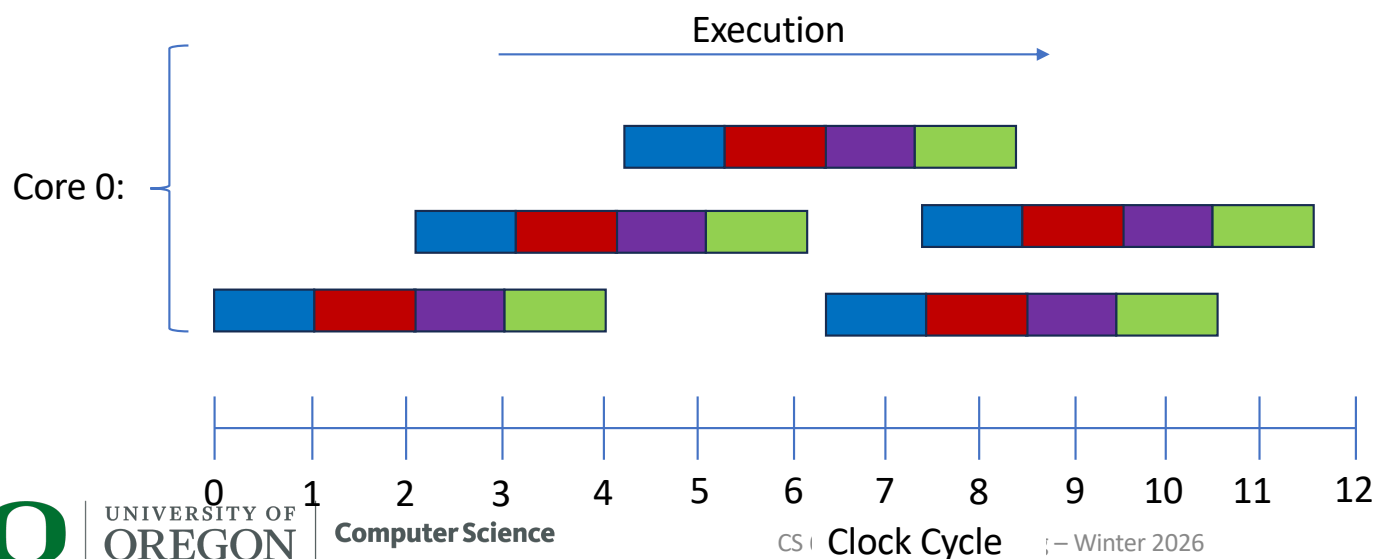
- Pipeline execution on the same core
 - Executing one instruction involves multiple stages on different HW components
 - Multiple executions can be executed at the same time they are pipelined at different stages



At any given cycle, no two instructions share the same stage

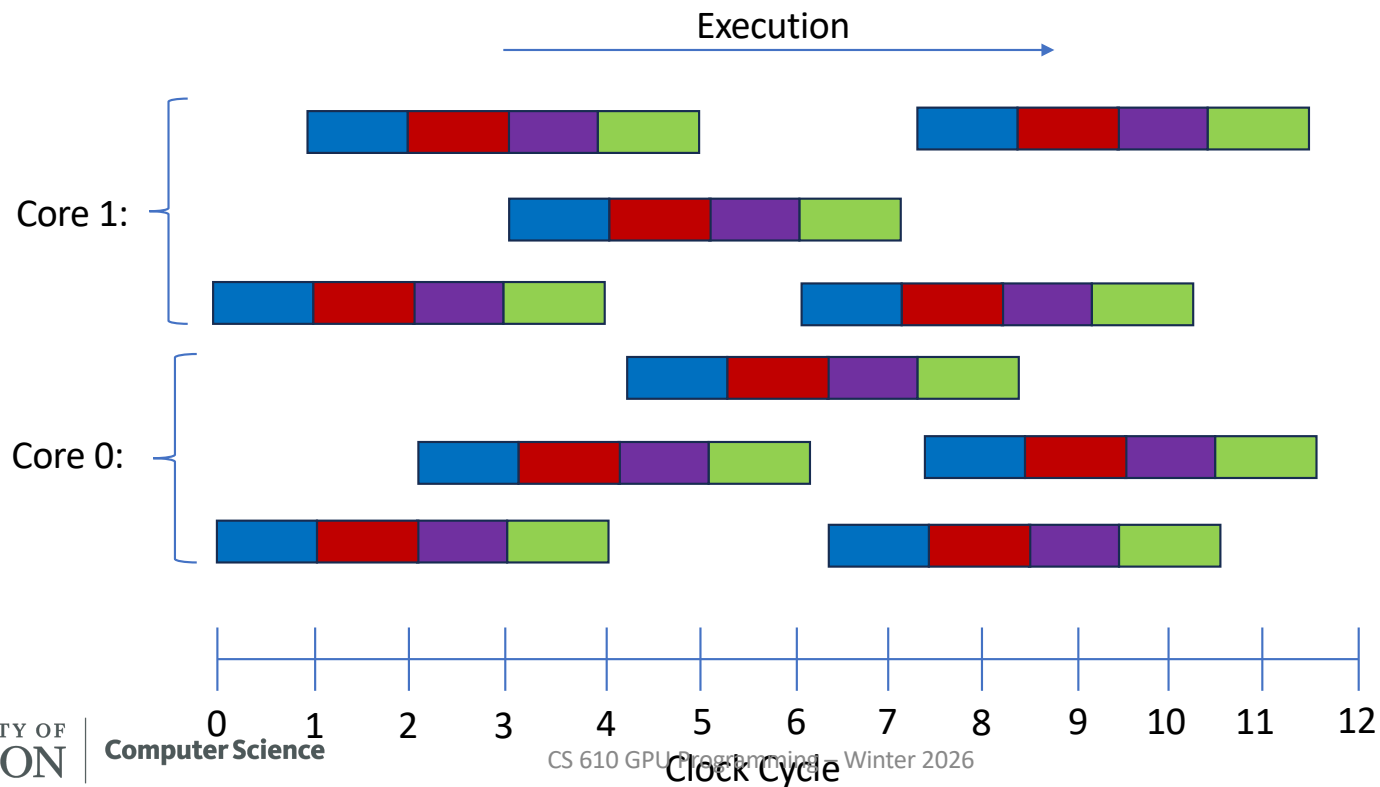
Why do instructions overlap?

- Pipeline execution on the same core
 - Executing one instruction involves multiple stages on different HW components
 - Multiple executions can be executed at the same time they are pipelined at different stages



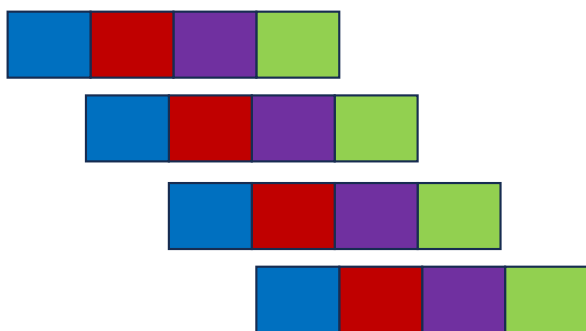
Why do instructions overlap?

- Multi-core parallelism + Pipeline execution

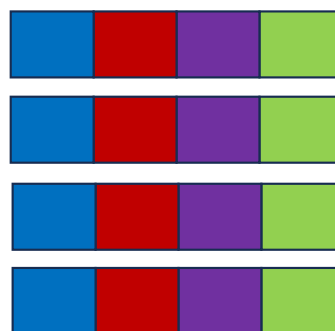


Concurrency vs. Parallelism

- Concurrency: multiple operations are happening at the same time
 - Boarder than parallelism



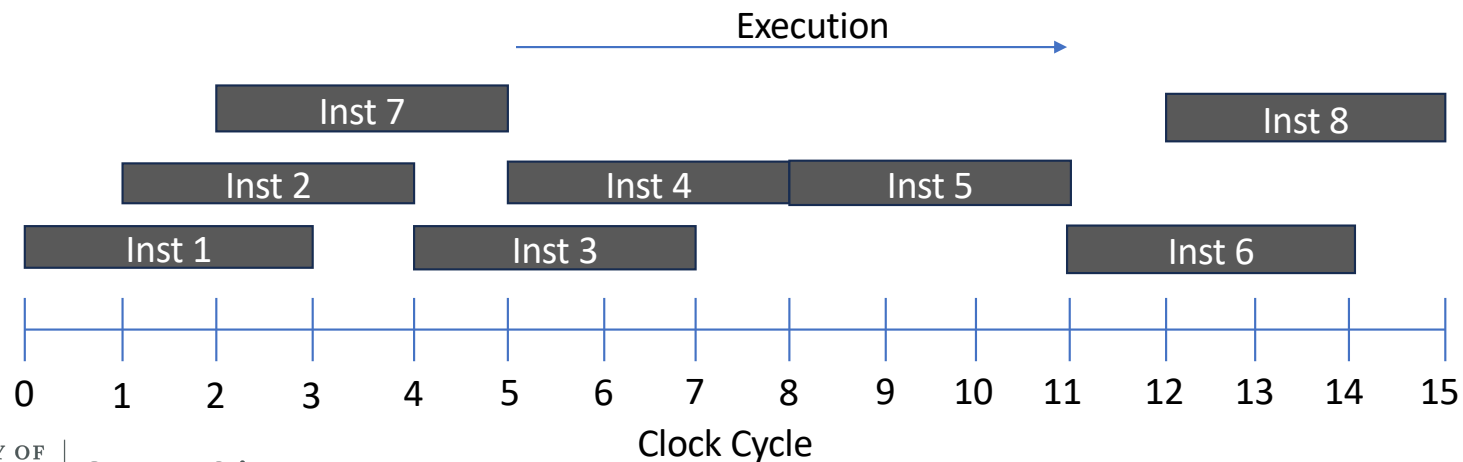
Concurrent
Not parallel



Concurrent
and parallel

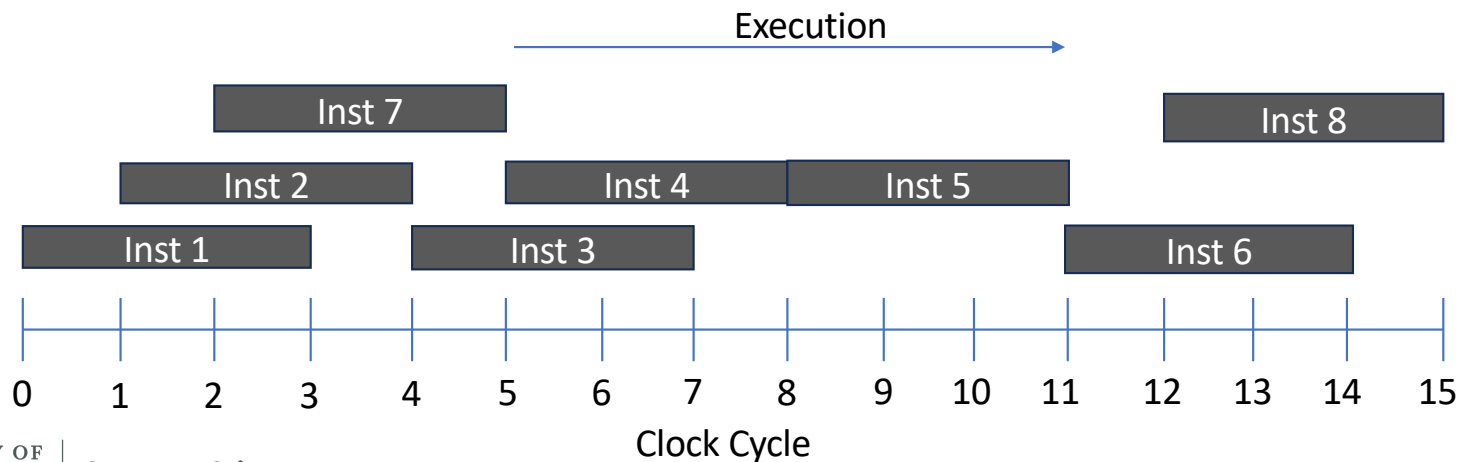
How do we measure the efficiency of a parallel program?

- Does your program use CUDA cores efficiently?
- Does your program use memory efficiently?



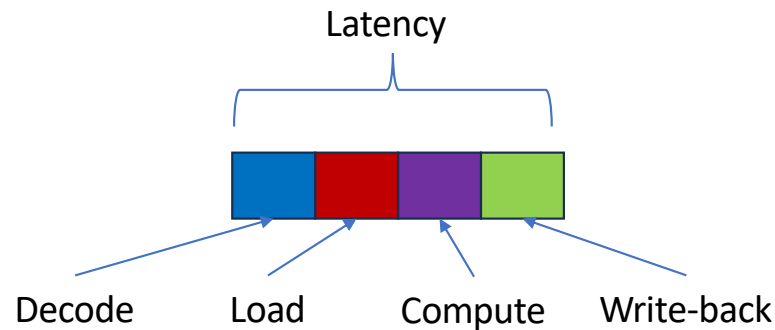
How do we measure the efficiency of a parallel program?

- First, we need to understand the concept of
 - Latency
 - Throughput



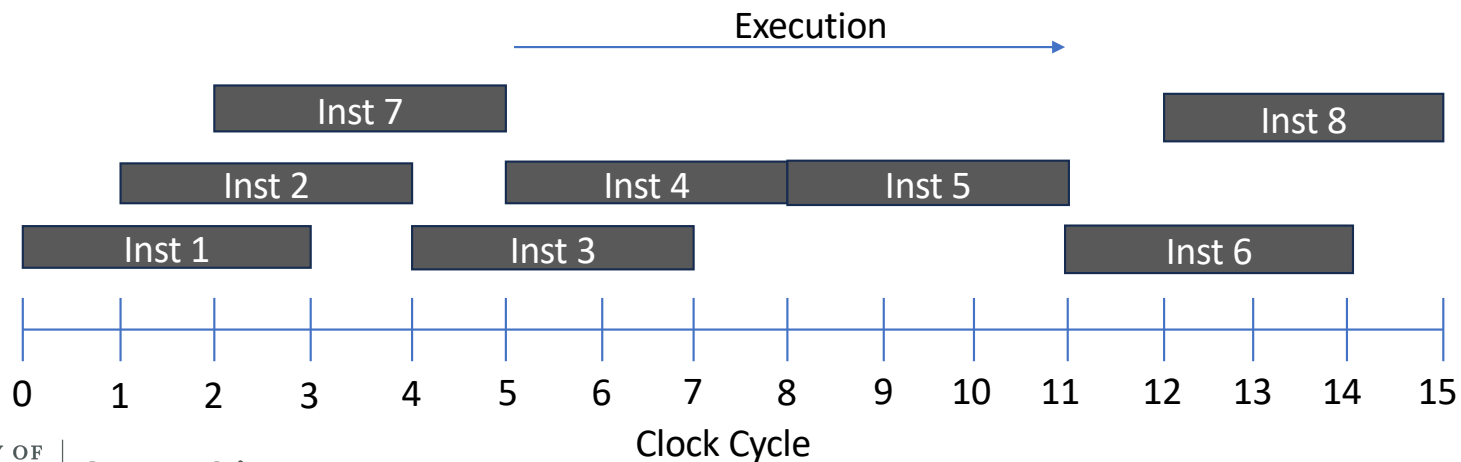
Latency

- The time between start and end of an item (e.g., instruction, task, etc.)
- Measured in time (e.g., # of cycles, second, etc.)
- Different instructions on different HW have different latencies



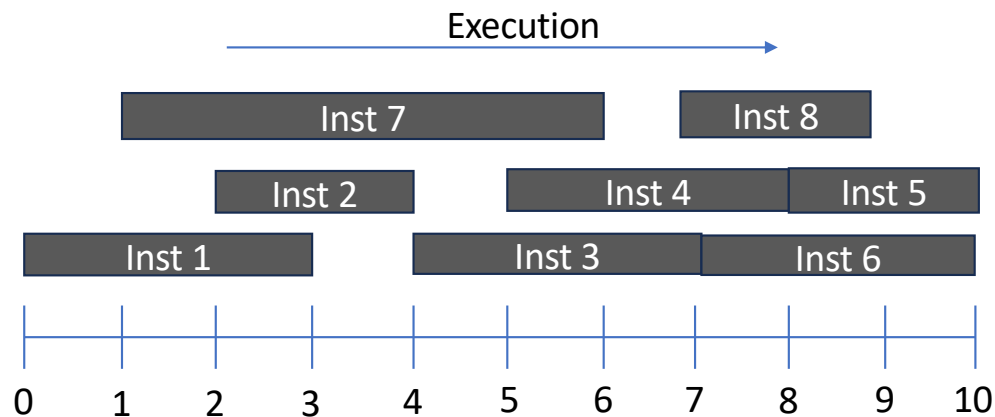
Latency

- The time between start and end of an item (e.g., instruction, task, etc.)
- Measured in time (e.g., # of cycles, second, etc.)
- What is the latency of each instruction in the example?



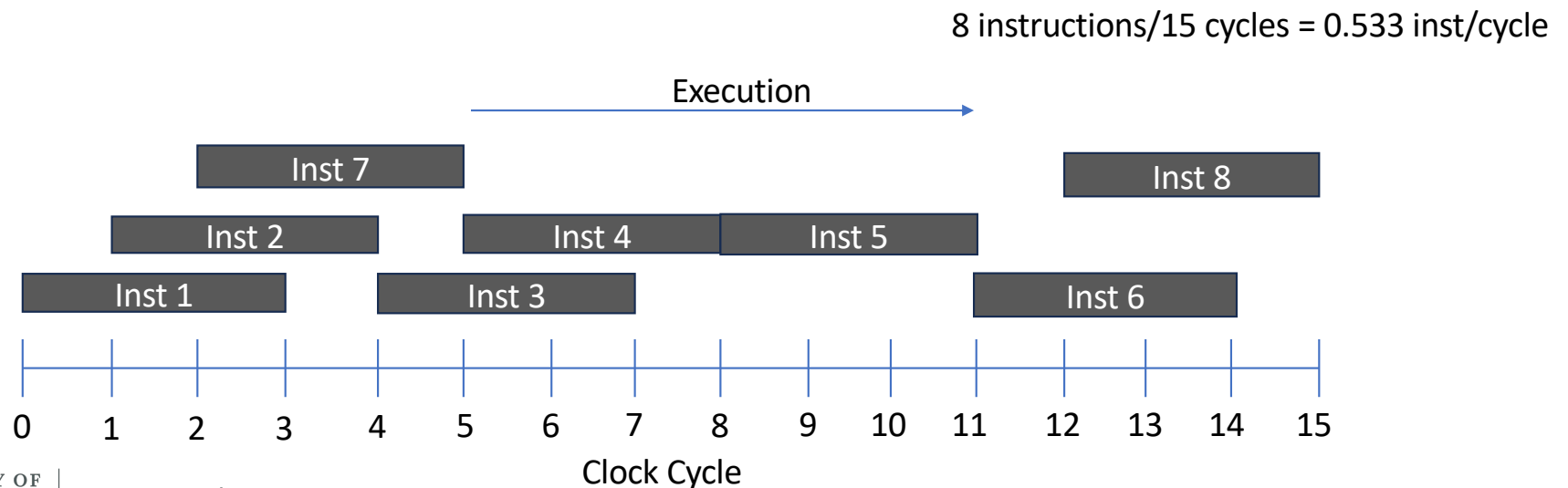
Latency

- The time between start and end of an item (e.g., instruction, task, etc.)
- Measured in time (e.g., # of cycles, second, etc.)
- What is the latency of each instruction in the example?



Throughput

- Number of items that can be processed given time interval divided by the duration of the interval
- Measured in # of items/unit time (e.g., inst/cycle, operations/s, etc.)
- What is the throughput of the following example?

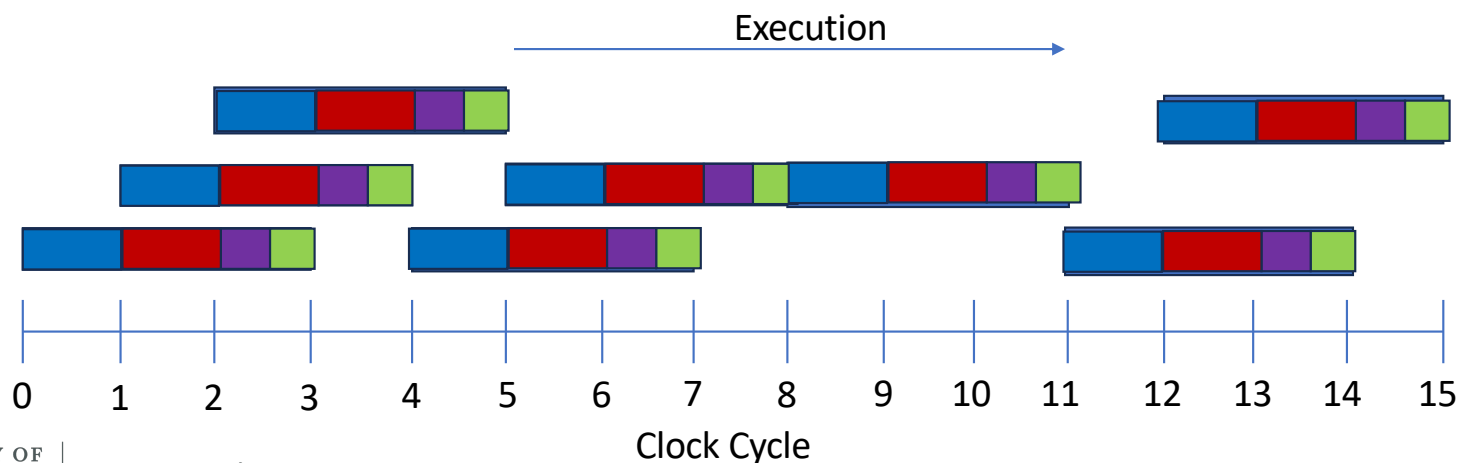


Throughput

- Number of items that can be processed given time interval divided by the duration of the interval
- Measured in # of items/unit time (e.g., inst/cycle, operations/s, etc.)
- What is the throughput of the following example?

■ : FP32 Add

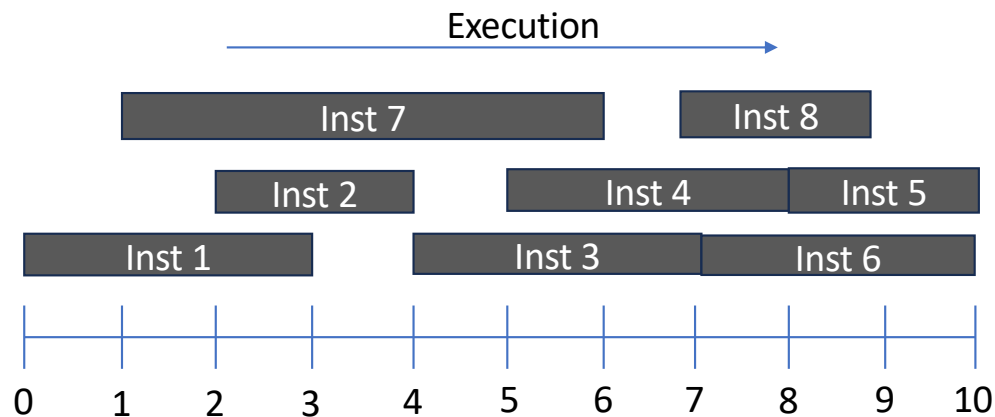
8 FP32 Add / 15 cycles = 0.533 FP32 Add/cycle



Throughput

- Number of items that can be processed given time interval divided by the duration of the interval
- Measured in # of items/unit time (e.g., inst/cycle, operations/s, etc.)
- What is the throughput of the following example?

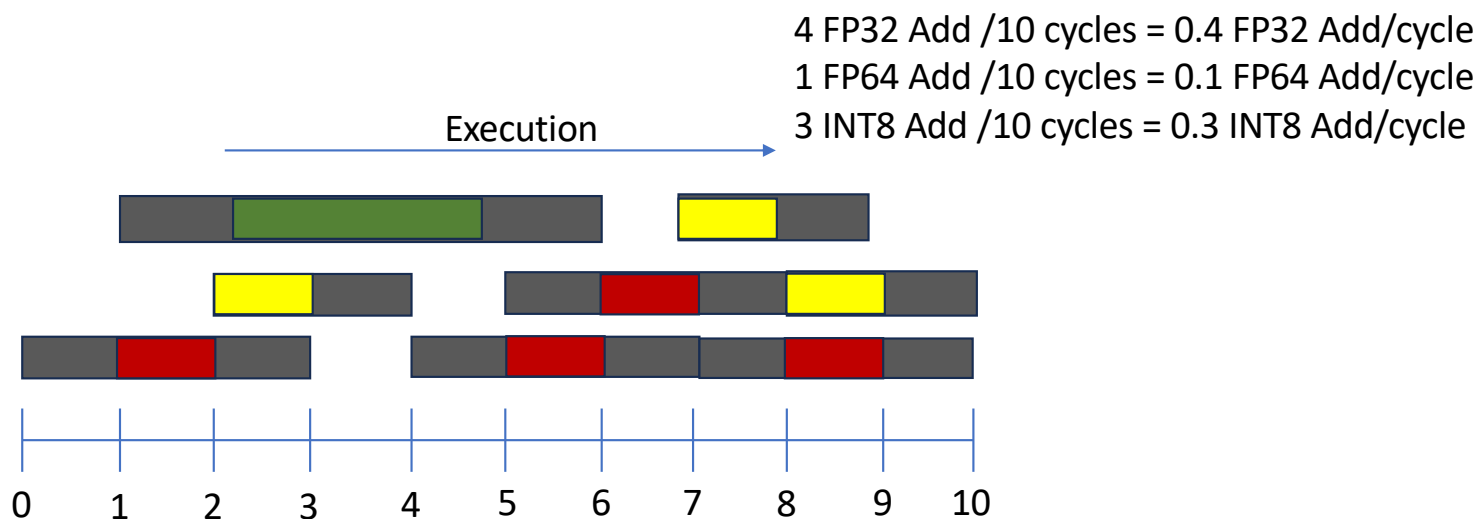
8 instructions/10 cycles = 0.8 inst/cycle



Throughput

- Number of items that can be processed given time interval divided by the duration of the interval
- Measured in # of items/unit time (e.g., inst/cycle, operations/s, etc.)
- What is the throughput of the following example?

■ : FP32 Add
■ : FP64 Add
■ : INT8 Add



How do we measure the efficiency of a parallel program?

- Throughput is one useful metric to measure the efficiency of our parallel program. E.g.:
 - Number of arithmetic operations done per second by CUDA core
 - Number of bytes read per second from memory
- Hardware has typical theoretical peak throughput
 - A100's FP32 peak operation throughput is 19.5 TFlops
 - A100's peak memory throughput is 1.555 TB/s
- Efficiency of a program

$$\text{Compute Efficiency} = \frac{\text{Achieved Arithmetic Throughput}}{\text{Theoretical Peak Arithmetic Throughput}}$$

$$\text{Memory Efficiency} = \frac{\text{Achieved Memory Throughput}}{\text{Theoretical Peak Memory Throughput}}$$

How do we measure the efficiency of a parallel program?

- Efficiency of a program

$$\text{Compute Efficiency} = \frac{\text{Achieved Arithmetic Throughput}}{\text{Theoretical Peak Arithmetic Throughput}}$$

$$\text{Memory Efficiency} = \frac{\text{Achieved Memory Throughput}}{\text{Theoretical Peak Memory Throughput}}$$

A100

- FP32 peak operation throughput: 19.5 TFlops
- Peak memory throughput: 1.555 TB/s

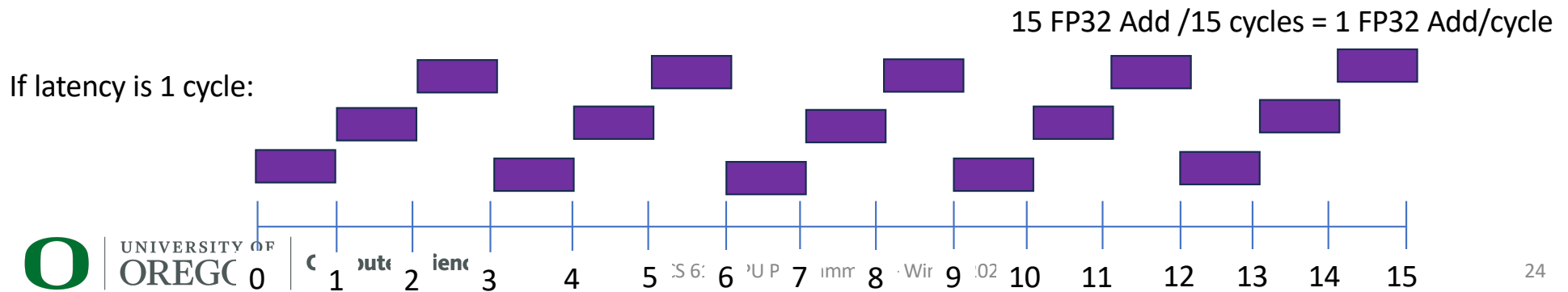
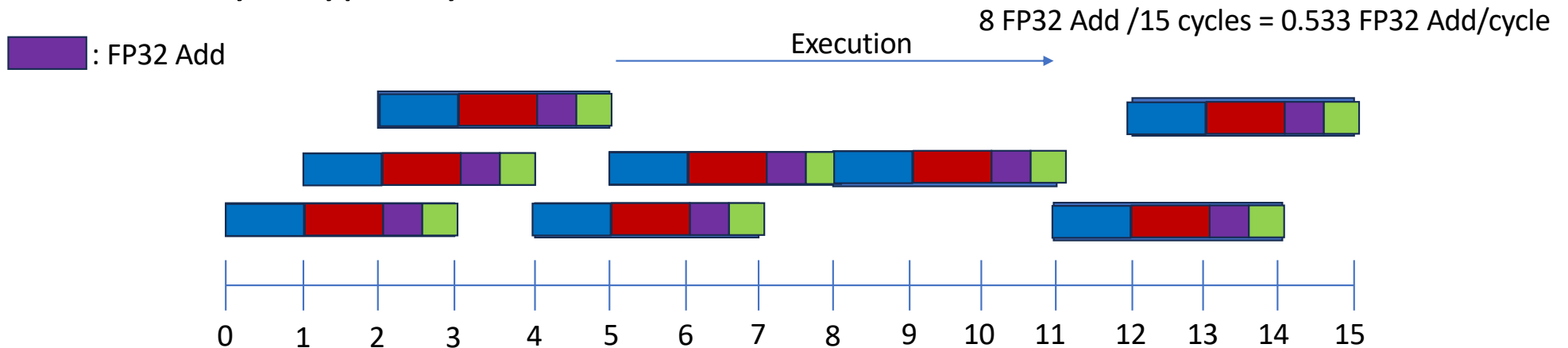
Example:

- A program achieved 10 TFlops FP32 throughput and 900GB/s memory throughput
- What is its compute and memory efficiency?

$$\text{Compute Efficiency} = \frac{10}{19.5} \approx 51\% \quad \text{Memory Efficiency} = \frac{0.9}{1.555} \approx 58\%$$

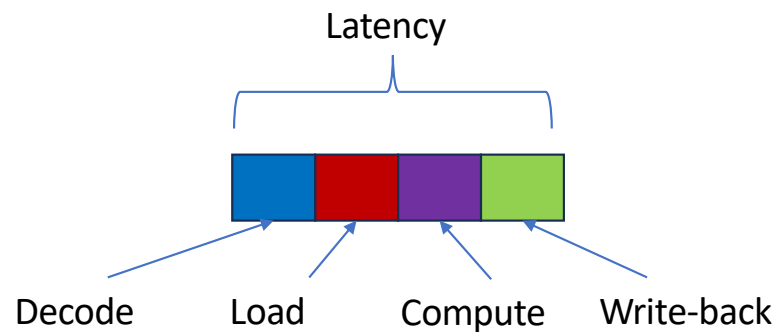
What is preventing us from achieving 100% efficiency?

- Latency is typically the main bottleneck



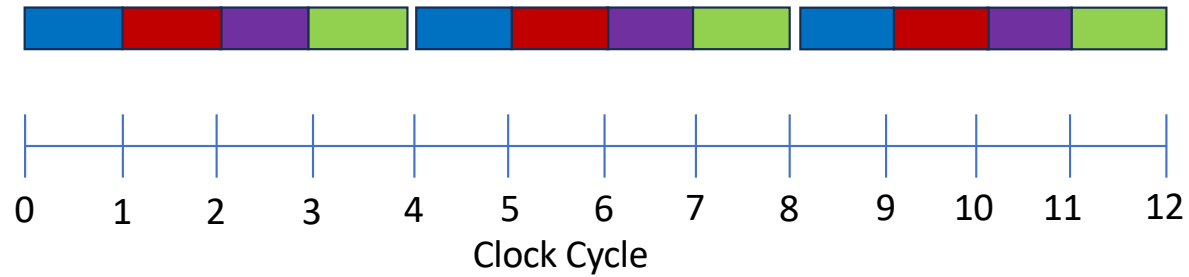
What is preventing us from achieving 100% efficiency?

- Latency is typically the main bottleneck
- However, the latency of an instruction is typically fixed. We cannot change it.
- What we can do: hide latency

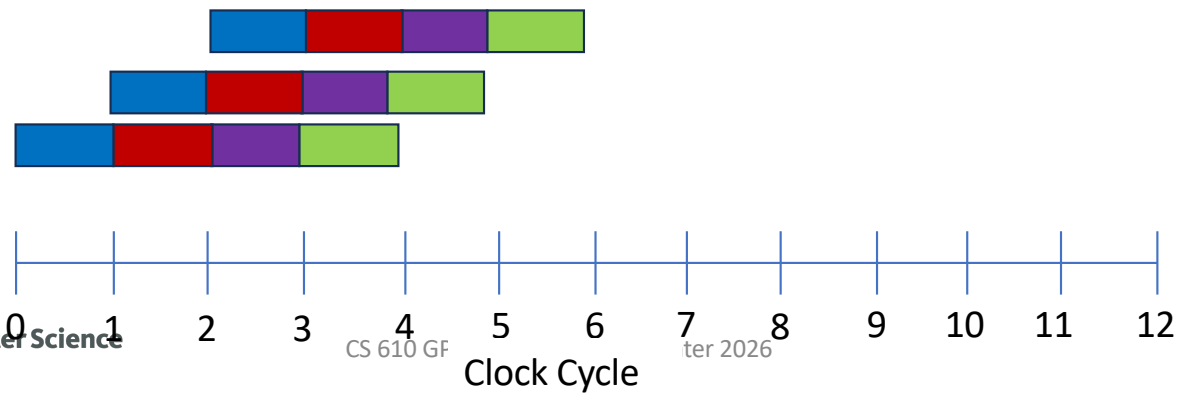


Latency hiding

Without latency hiding

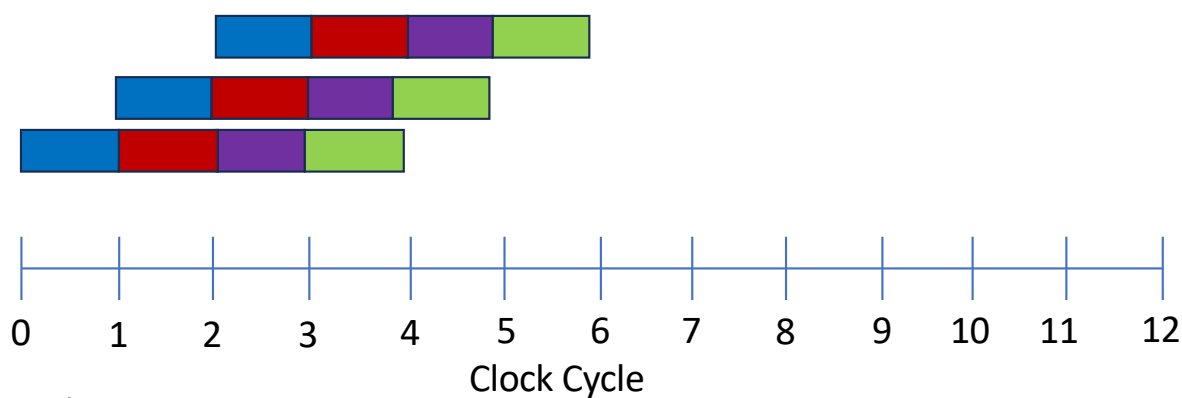


With latency hiding



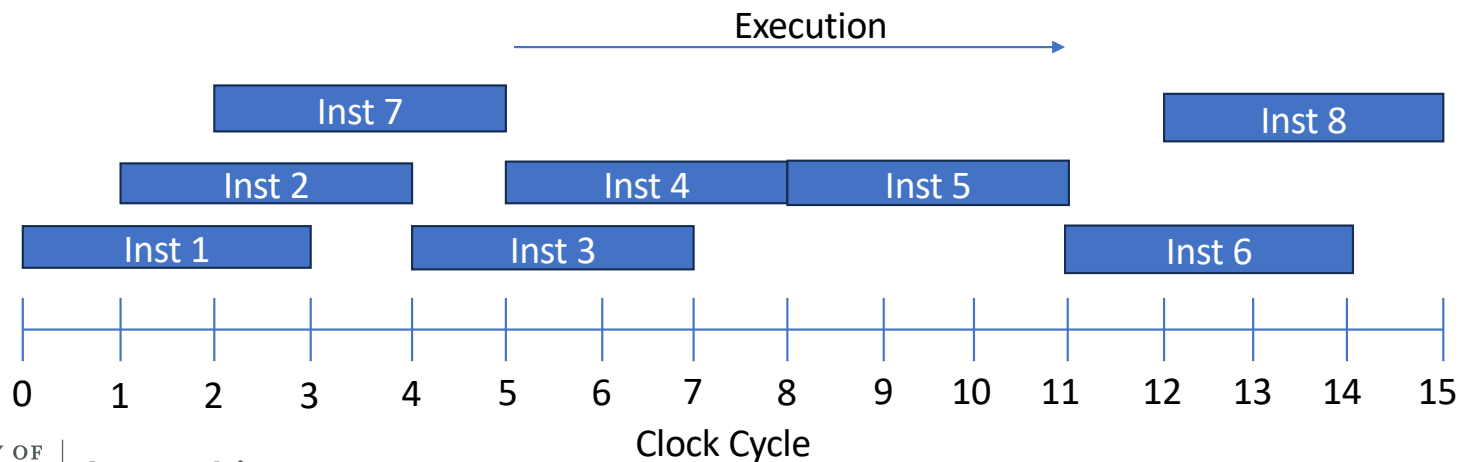
How to achieving latency hiding?

- Degree of concurrency
= Number of items processed at the same time
(a moment of time or average over the interval)
- Measured in # of items
- What is the concurrency of within the 2nd cycle?



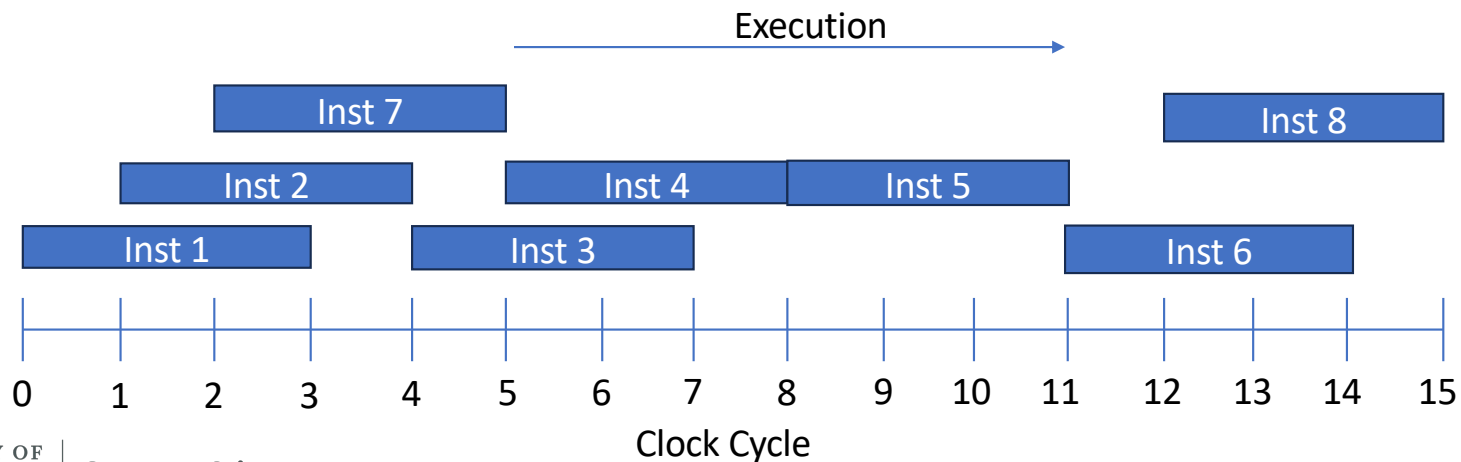
Concurrency

- Number of items processed at the same time
 - a moment of time or average over the interval
- Measured in # of items
- What is the concurrency of within the 2nd cycle?



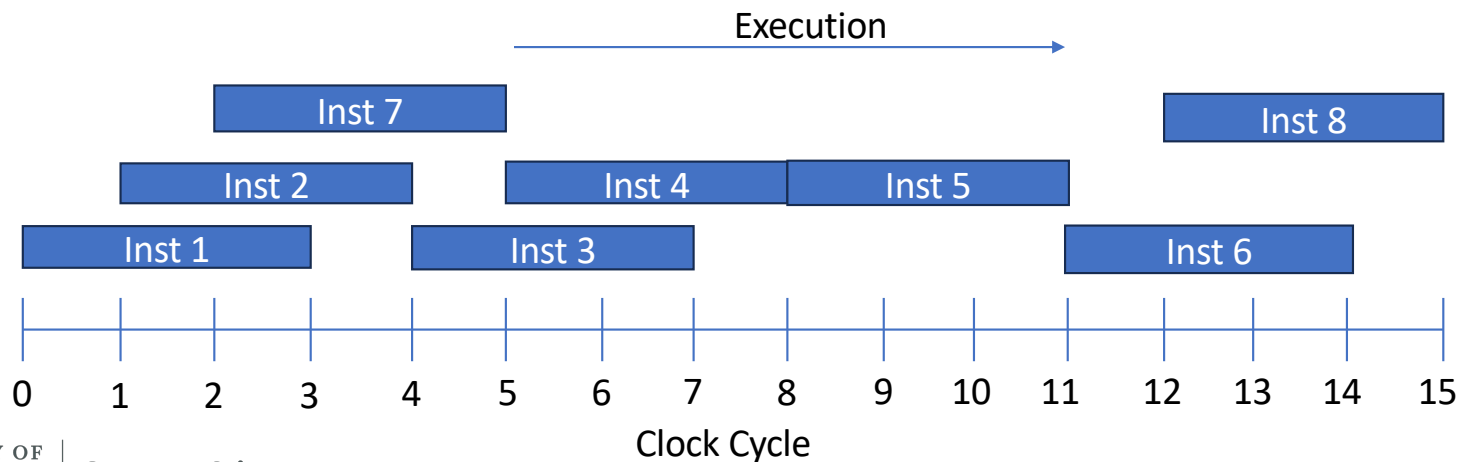
Concurrency

- Number of items processed at the same time
 - a moment of time or average over the interval
- Measured in # of items
- What is the concurrency of within the 6th cycle?



Concurrency

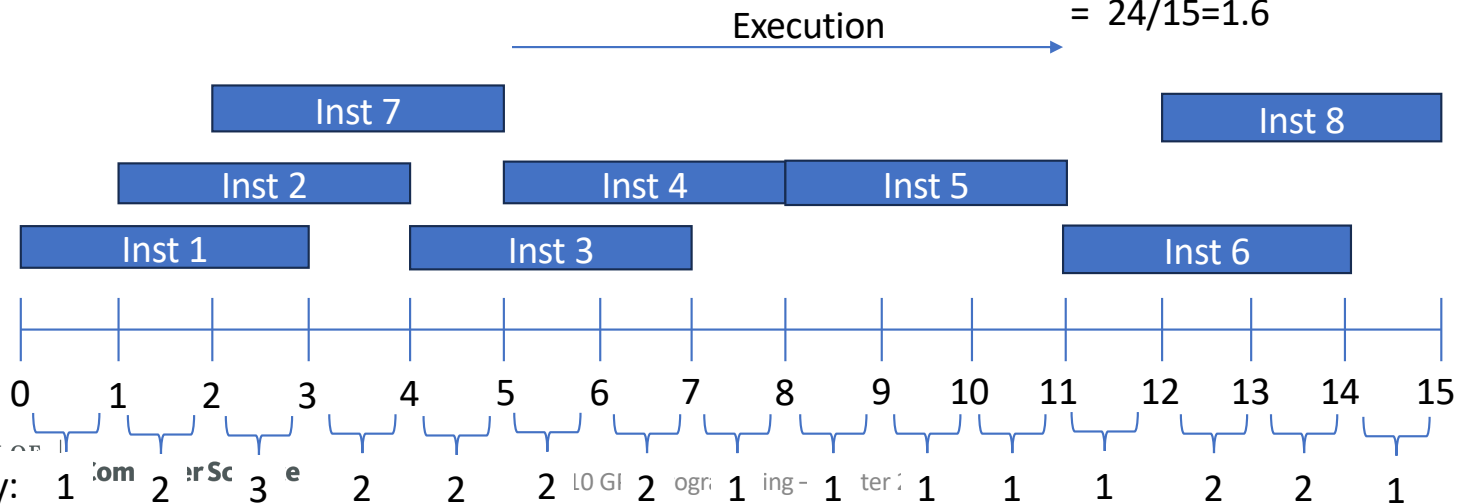
- Number of items processed at the same time
 - a moment of time or average over the interval
- Measured in # of items
- What is the concurrency of within the 11th cycle?



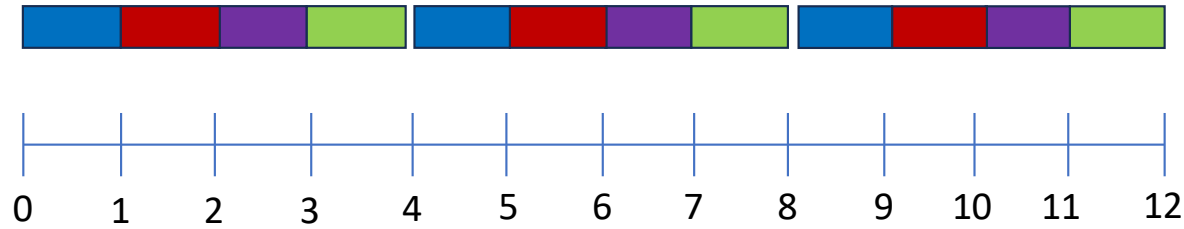
Concurrency

- Number of items processed at the same time
 - a moment of time or average over the interval
- Measured in # of items
- What is the average concurrency for total 15 cycles?

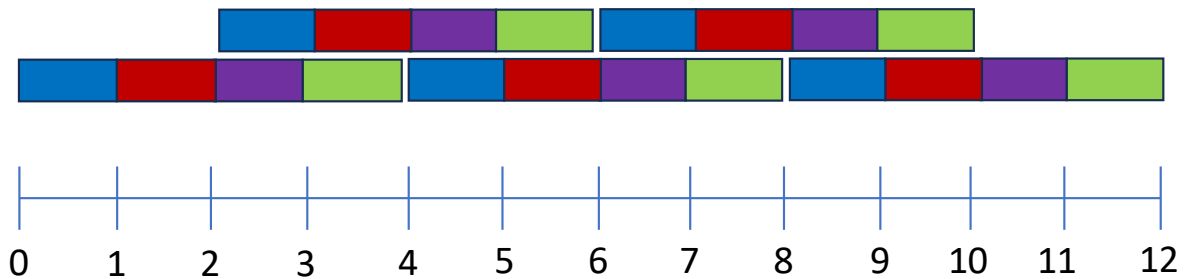
Avg. Concurrency =
 $(1+2+3+2+2+2+2+1+1+1+1+1+2+2+1)/15$
 $= 24/15=1.6$



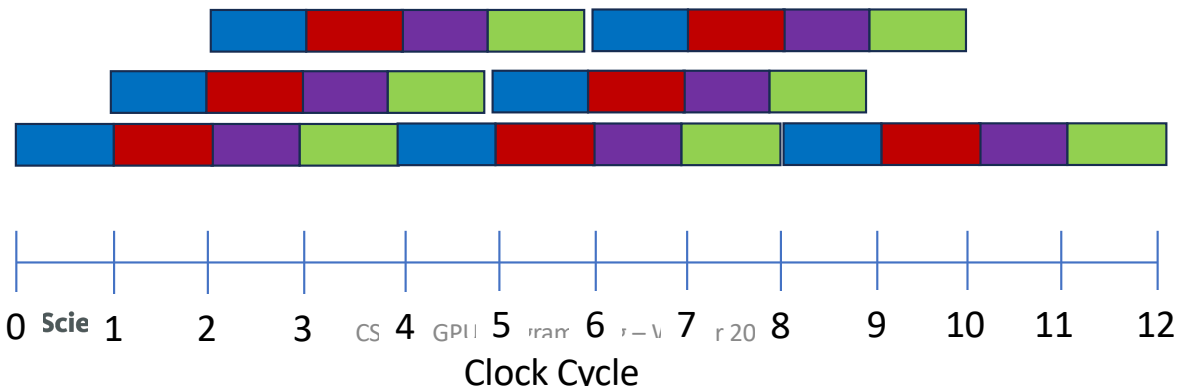
Relations between concurrency and throughput



throughput =
 $3/12=0.25$ inst/cycle



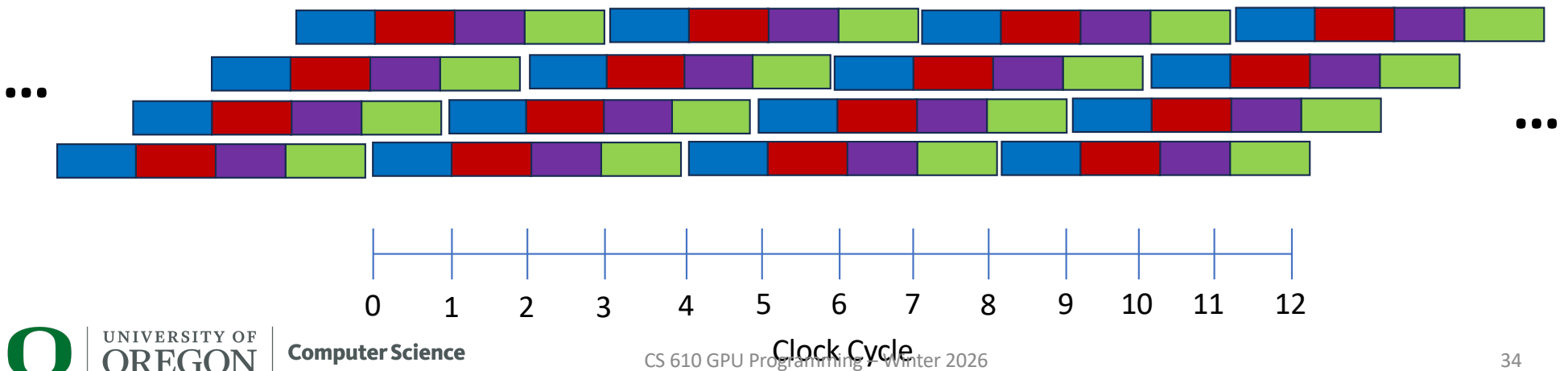
throughput =
 $5/12=0.42$ inst/cycle



throughput =
 $5/12=0.58$ inst/cycle

Relations between concurrency and throughput

- Higher concurrency leads to better latency hiding and higher throughput
- Maximum/Peak throughput is achieved when latency hiding is maximized. But this does not necessarily mean latency is completely hidden



Little's law

- Latency of a type of instruction is constant (HW dependent)
- Throughput changes with concurrency

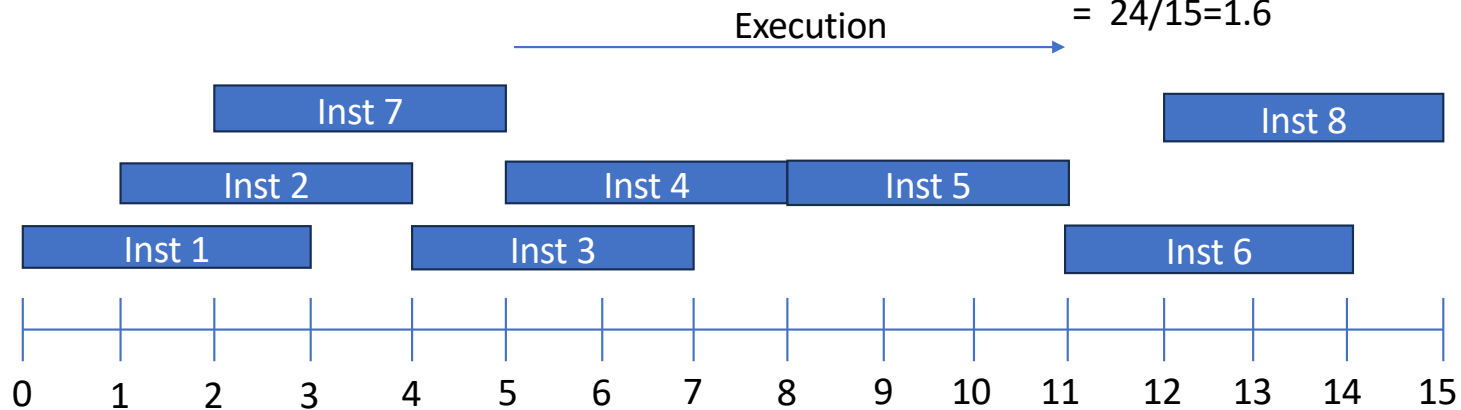
$$\textit{Throughput} = \frac{\textit{Concurrency}}{\textit{Latency}}$$

Little's law for same-type instruction

- All instructions have same latency: 3 cycles

$$\text{Throughput} = \frac{\text{Average Concurrency}}{\text{Latency}} = \frac{1.6}{3} = 0.533 \text{ inst/cycle}$$

$$\begin{aligned} \text{Avg. Concurrency} &= \\ &= (1+2+3+2+2+2+2+1+1+1+1+1+2+2+1)/15 \\ &= 24/15=1.6 \end{aligned}$$

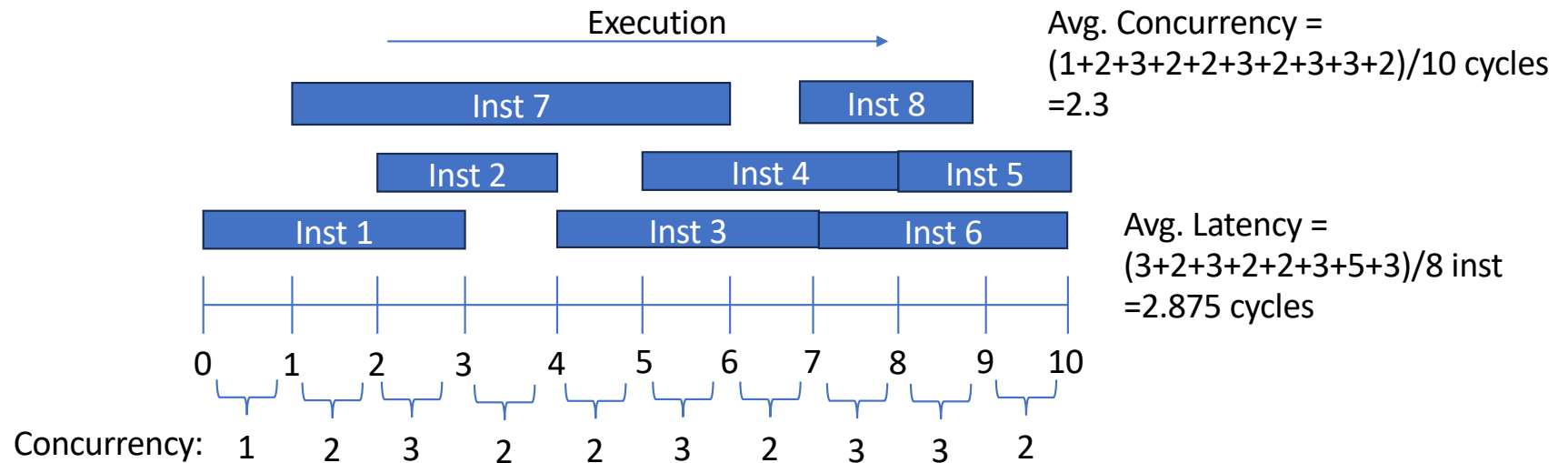


c Same instruction latency

Little's law for mixed-type instructions

- Instructions have different latencies

$$\text{Throughput} = \frac{\text{Average Concurrency}}{\text{Average Latency}} = \frac{2.3}{2.875} = 0.8 \text{ inst/cycle}$$



How does little's law help us?

- Little's law tell us how much concurrency we need to achieve for getting certain throughput

$$\textit{Throughput} = \frac{\textit{Concurrency}}{\textit{Latency}}$$



$$\textit{Concurrency} = \textit{Latency} * \textit{Throughput}$$

Example

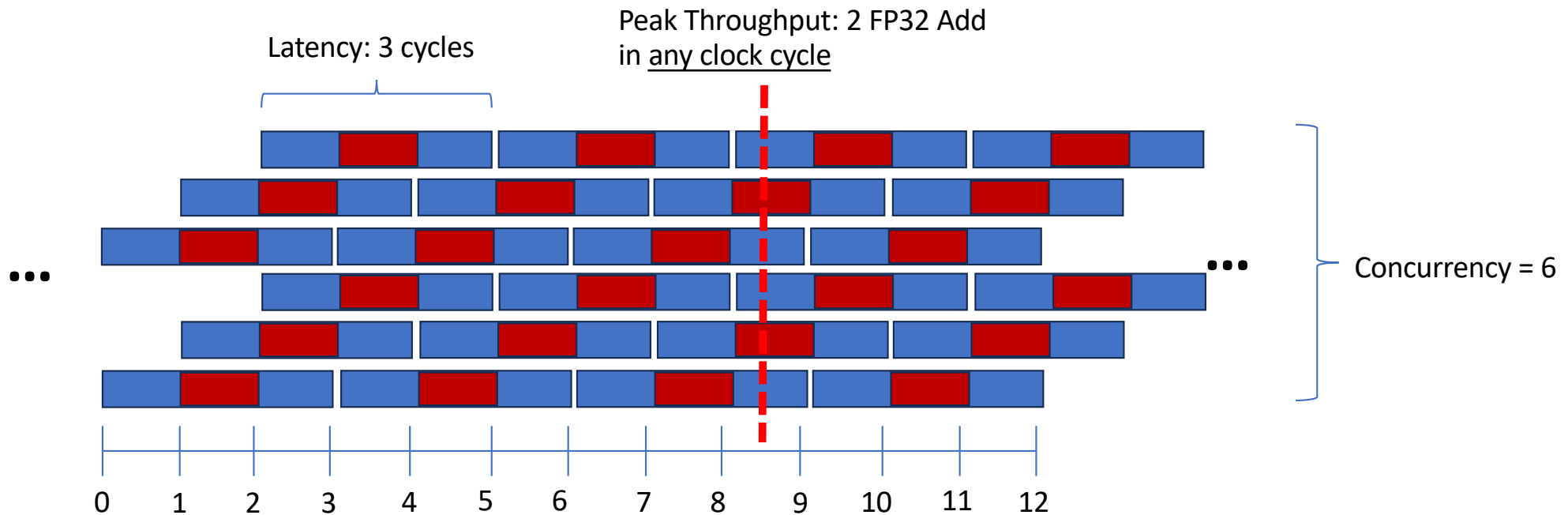
- A system's theoretical peak FP32 Add throughput is 2 IPC. Each FP32 Add instruction has 3 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?
- IPC = instruction per cycle

$$\text{Concurrency} = \text{Latency} * \text{Throughput} = 3 * 2 = 6$$

We need 6 instructions executing at the same time to be able to full hide the latency and achieve peak throughput

Example

- A system's theoretical peak FP32 Add throughput is 2 IPC. Each FP32 Add instruction has 3 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?



Example

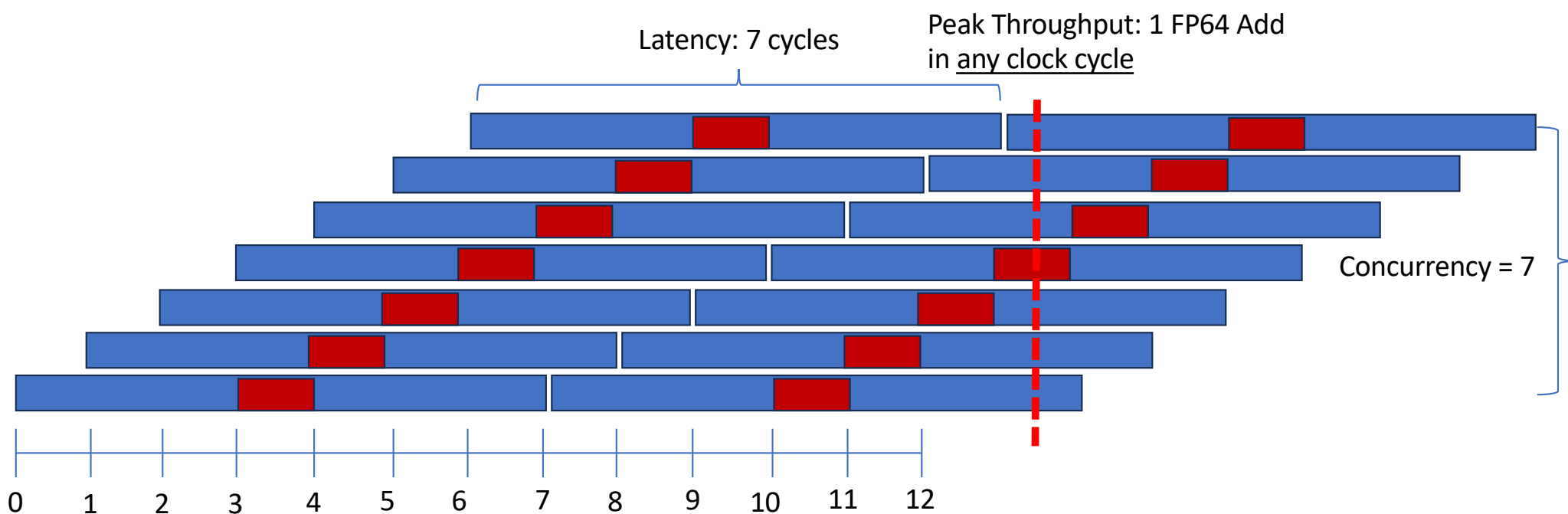
- A system's theoretical peak FP64 Add throughput is 1 IPC. Each FP64 Add instruction has 7 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?

$$\text{Concurrency} = \text{Latency} * \text{Throughput} = 7 * 1 = 7$$

We need 7 instructions executing at the same time to be able to full hide the latency and achieve peak throughput

Example

- A system's theoretical peak FP64 Add throughput is 1 IPC. Each FP64 Add instruction has 7 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?



Example

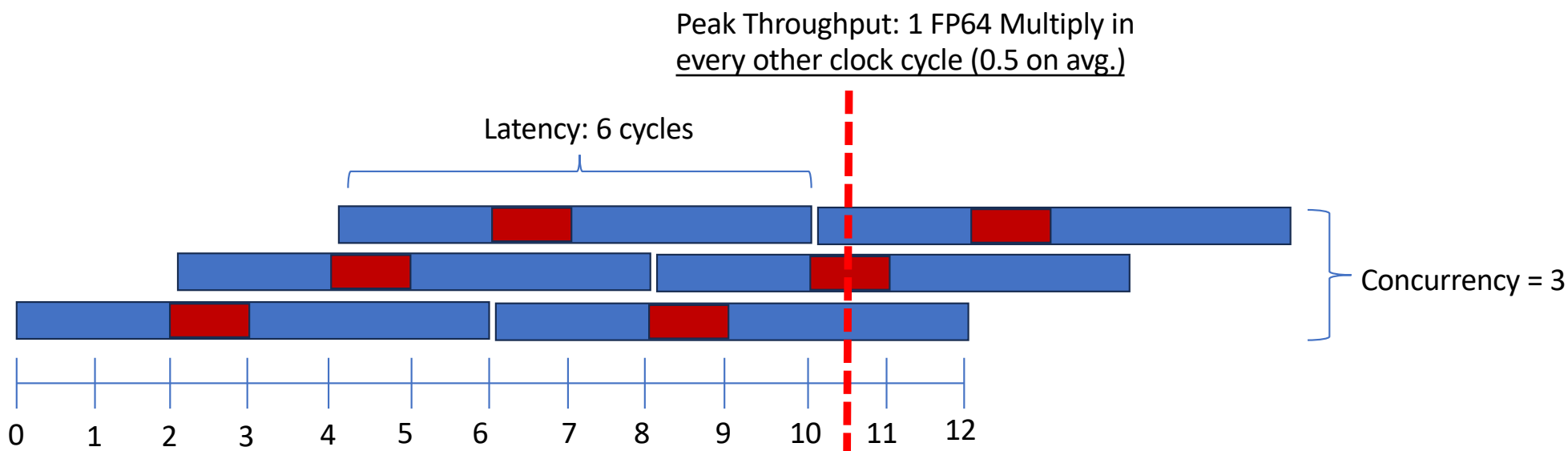
- A system's theoretical peak FP64 Multiply throughput is 0.5 IPC. Each FP64 Multiply instruction has 6 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?

$$\textit{Concurrency} = \textit{Latency} * \textit{Throughput} = 6 * 0.5 = 3$$

We need 3 instructions executing at the same time to be able to full hide the latency and achieve peak throughput

Example

- A system's theoretical peak FP64 Multiply throughput is 0.5 IPC. Each FP64 Multiply instruction has 6 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?



Example

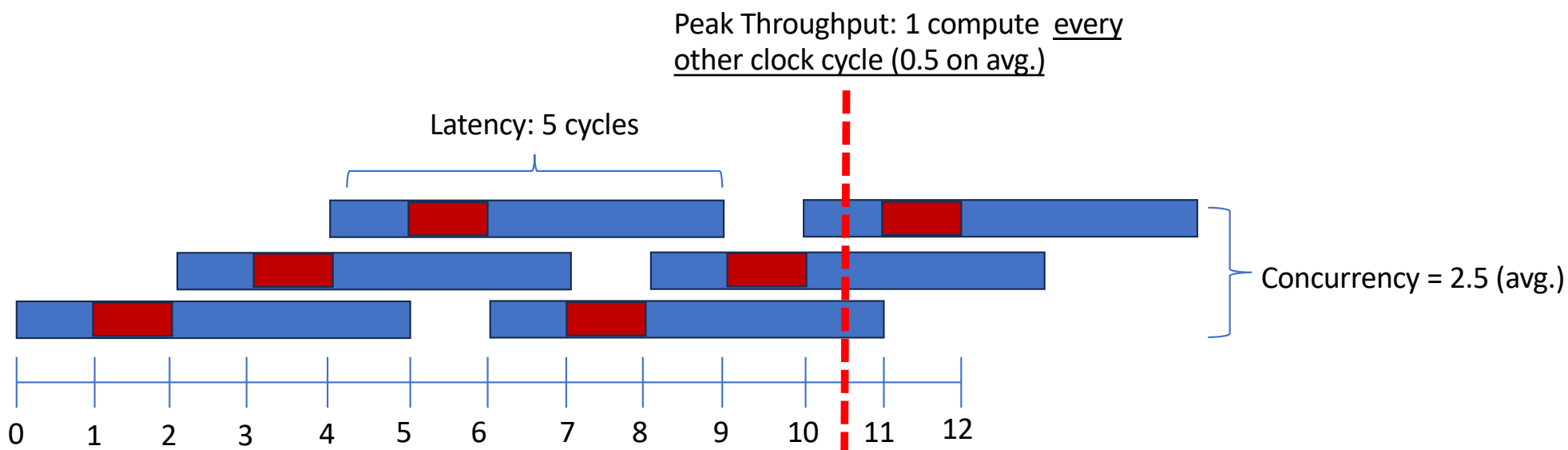
- A system's theoretical peak throughput of instruction Y is 0.5 IPC. Each instruction Y has 5 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?

$$\text{Concurrency} = \text{Latency} * \text{Throughput} = 5 * 0.5 = 2.5$$

We need 2.5 instructions executing at the same time to be able to full hide the latency and achieve peak throughput

Example

- A system's theoretical peak throughput of instruction Y is 0.5 IPC. Each instruction Y has 5 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?



Example

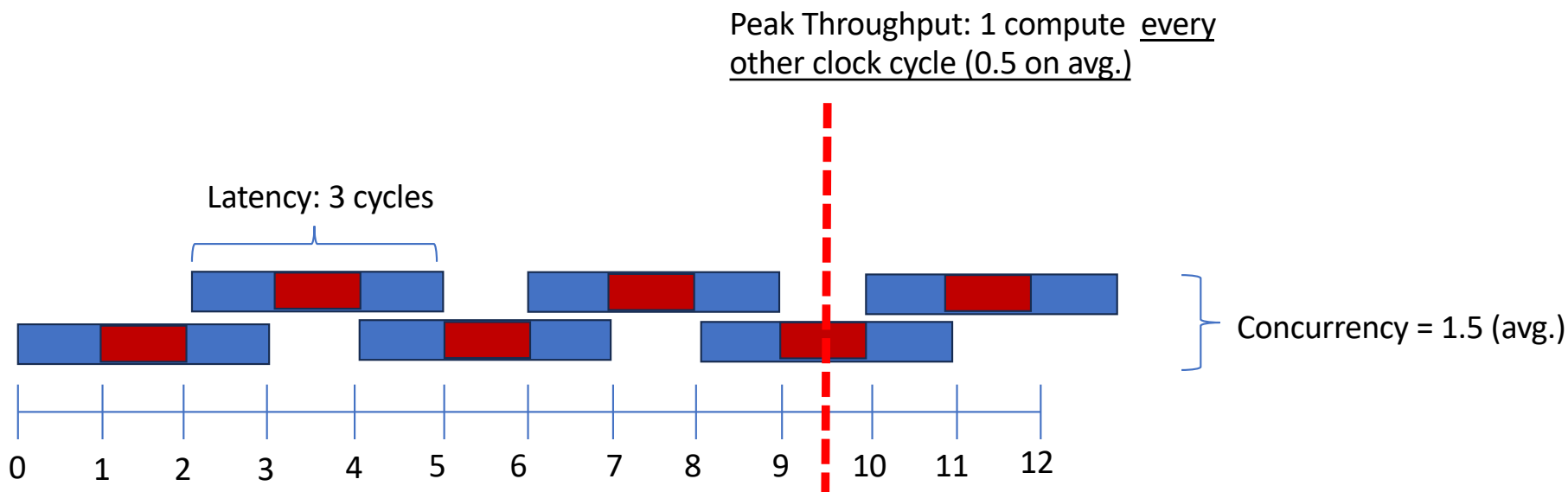
- A system's theoretical peak throughput of instruction X is 0.5 IPC. Each instruction X has 3 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?

$$\text{Concurrency} = \text{Latency} * \text{Throughput} = 3 * 0.5 = 1.5$$

We need 1.5 instructions executing at the same time to be able to full hide the latency and achieve peak throughput

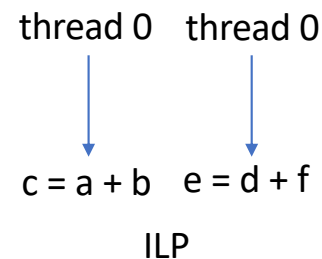
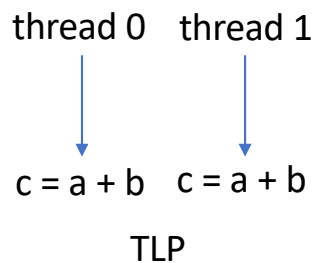
Example

- A system's theoretical peak throughput of instruction X is 0.5 IPC. Each instruction X has 3 cycle latency. How much concurrency is needed for achieving the theoretical peak throughput?
- Maximum/Peak throughput is achieving when latency hiding is maximized. But this does not necessarily mean latency is completely hidden

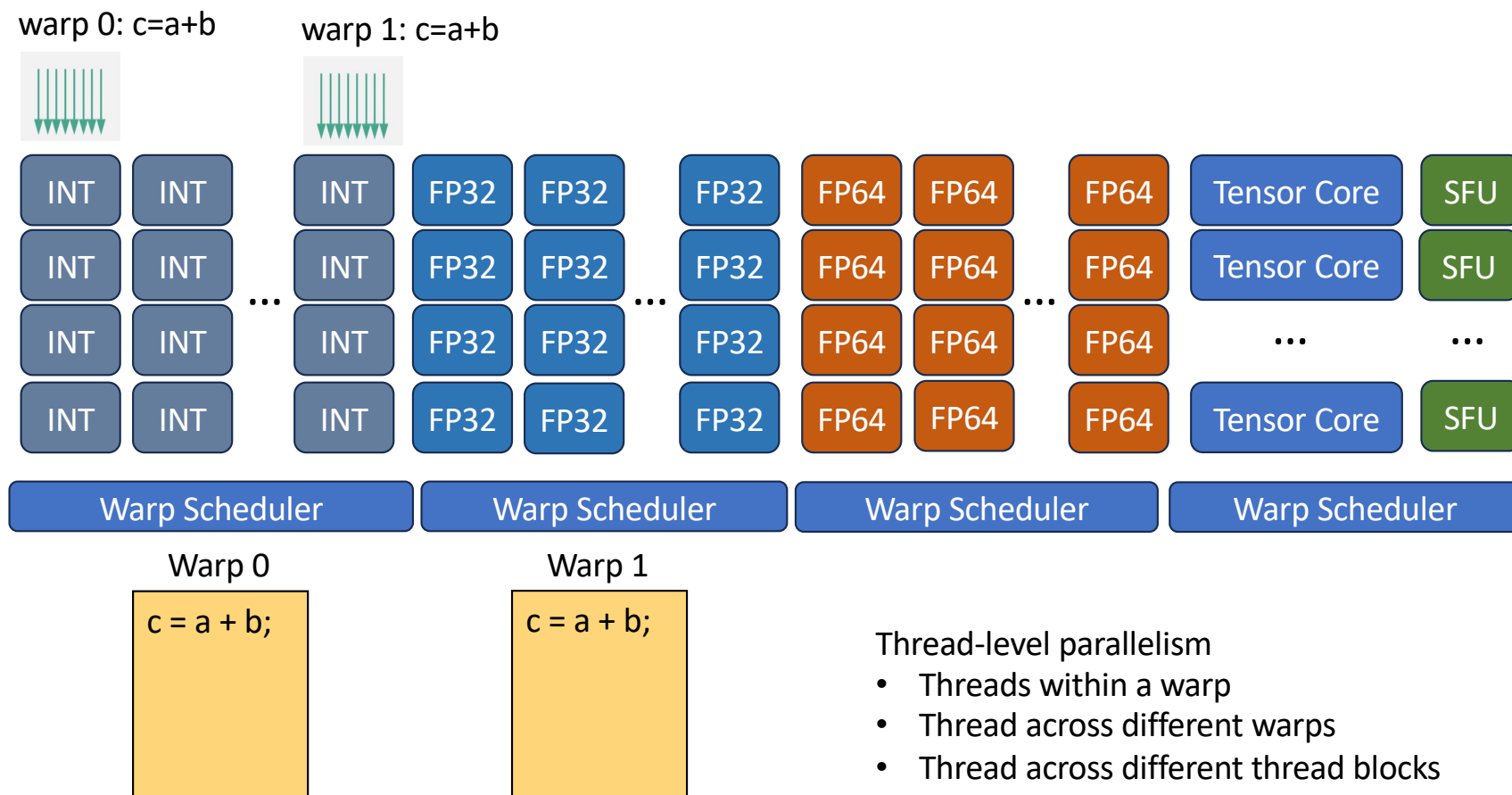


Concurrency in GPU

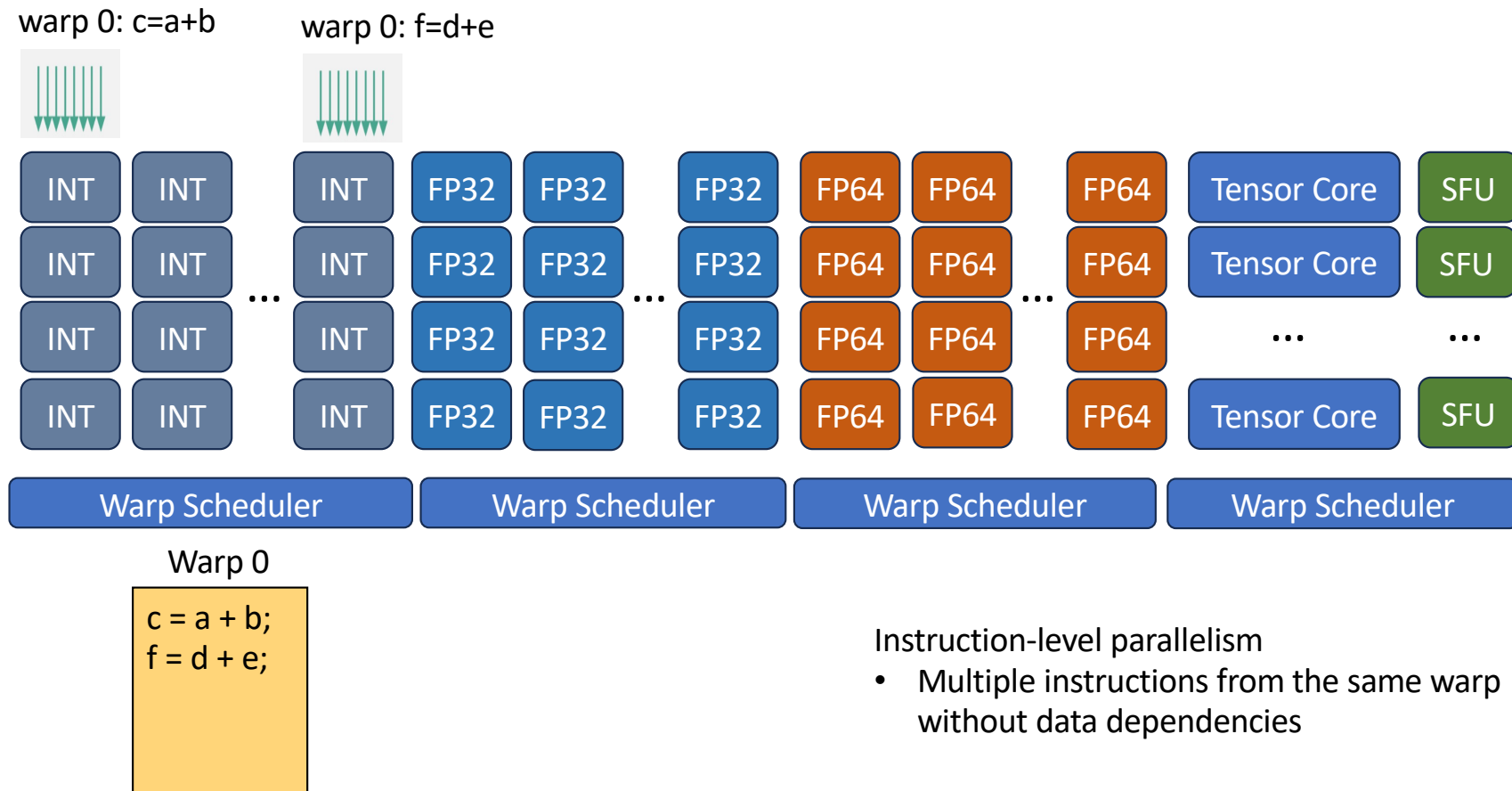
- Thread-level parallelism (TLP)
 - Threads within a warp
 - Thread across different warps
 - Thread across different thread blocks
- Instruction-level parallelism (ILP)
 - Instructions from the same warp do not have data dependencies can be executed concurrently



Thread-level parallelism (TLP)



Instruction-level parallelism (ILP)



How do warp schedulers manage warps?

- Each SM has 2-4 warp schedulers
 - Each works independently
 - Each manages several warps (from same or different thread blocks)
- At every clock cycle each warp scheduler will try to
 1. Find an instruction from a warp that is ready to be executed
 2. Issue (i.e., execute) the instruction by assigning it to CUDA cores
- A warp scheduler **does not need to wait** for an instruction to finish execute before it can issue another instruction.
- For issuing instructions from a warp scheduler
 - Latency: 1 clock cycle
 - Throughput
 - 1 IPC for most GPUs
 - 2 IPC for GPU with sm_60 (two instructions per cycle)



When does an instruction become ready?

- All input operands are in registers
- All data dependencies are met
- Following the execution order enforced by memory fence/ synchronization
- Resource/Cores are available

```
Inst 1: a = a1+a2;  
Inst 2: b = b1+b2;  
Inst 3: c = a+b;
```

Inst 3 cannot be ready
until Inst 1&2 are finished

```
Inst 1: a = a1+a2;  
Inst 2: b = b1+b2;  
Inst 3: c = c1+c2;
```

Inst 3 can be ready without
waiting for Inst 1&2

```
Inst 1: a ← Load from memory  
Inst 2: b ← Load from memory  
Inst 3: c = a+b;
```

Inst 3 cannot be ready until Inst
1&2 are finished

```
Inst 1: A[10]++;  
Inst 2: memory fence  
Inst 3: a = A[10];
```

Inst 3 cannot be ready
until Inst 1&2 are finished

```
Inst 1: shared_mem[i] = A[i]  
Inst 2: __syncthreads();  
Inst 3: a = A[i] + A[i+1];
```

Inst 3 cannot be ready until
synchronization finished

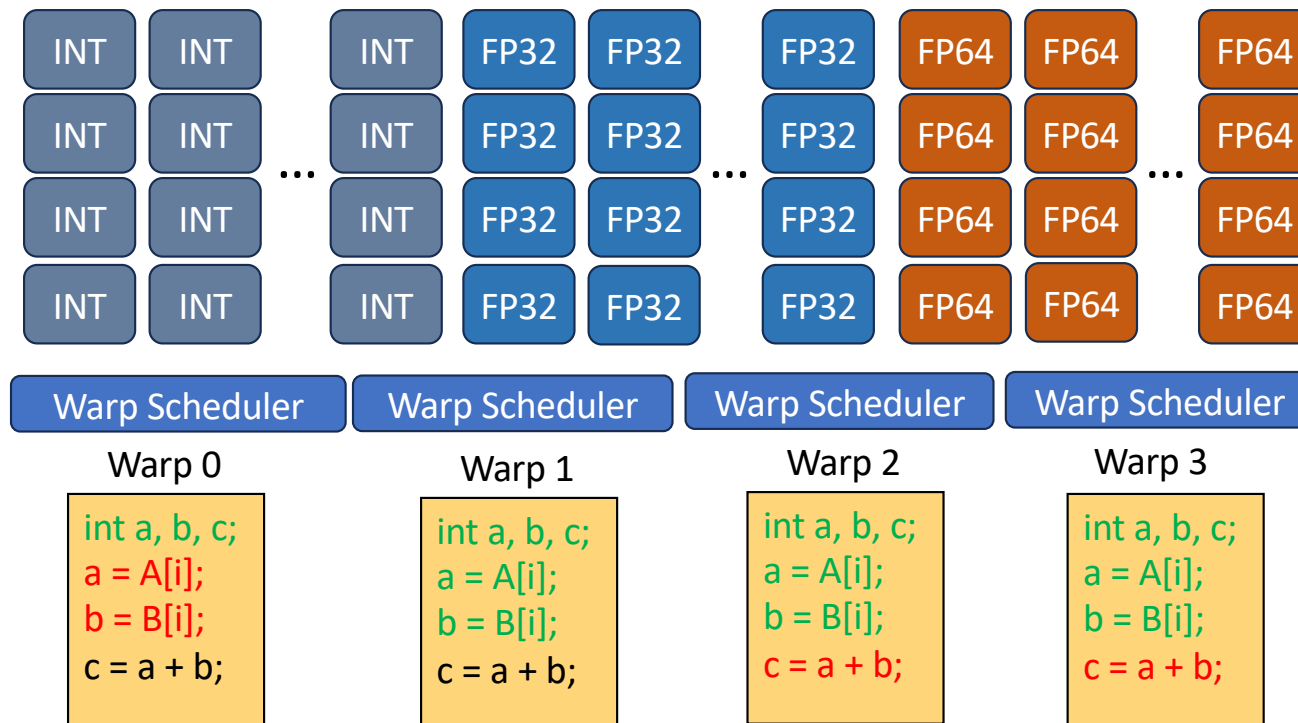


At each instruction issue time, which warp can be chosen?

- Any warp with a ready instruction
 - Can be different than any executing warp → thread level parallelism
 - Can be same as any executing warp → instruction level parallelism

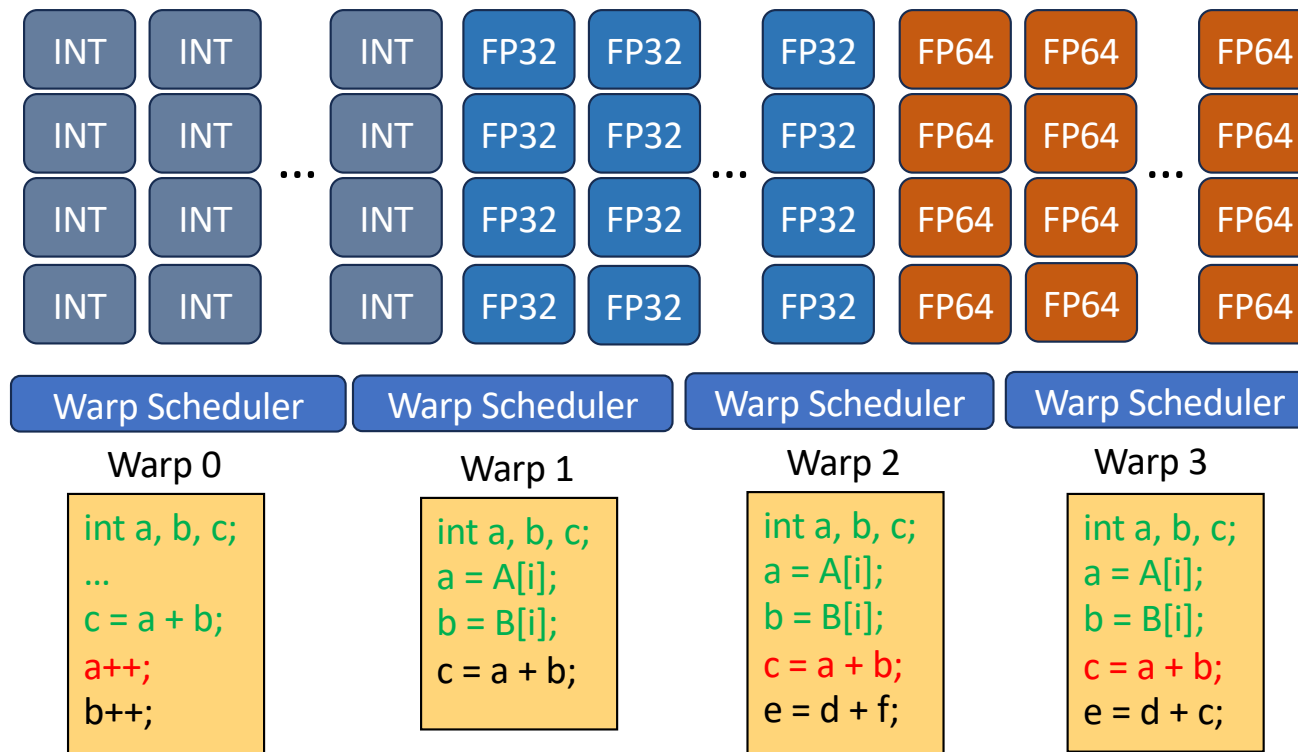
Which instruction/warp can be issued next?

Assuming the computing throughput is unlimited → Cores can always take new instructions



Which instruction/warp can be issued next?

Assuming the computing throughput is unlimited → Cores can always take new instructions



Completed : green
Executing : red
To execute: black

Measuring throughput for warp

- A warp of threads (32) execute the same instruction in a cycle
- 32 Instruction per second (IPC) = 1 Warp instruction per second (WIPC)

Compute Capability	5.0, 5.2	5.3	6.0	6.1	6.2	7.x	8.0	8.6	8.9	9.0
16-bit floating-point add, multiply, multiply-add	N/A	256	128	2	256	128	256 ³		128	256
32-bit floating-point add, multiply, multiply-add	128		64	128		64		128		
64-bit floating-point add, multiply, multiply-add	4		32	4		32 ⁵	32	2		64
32-bit floating-point reciprocal, reciprocal square root, base-2 logarithm (<code>__log2f</code>), base 2 exponential (<code>exp2f</code>), sine (<code>__sinf</code>), cosine (<code>__cosf</code>)	32		16	32		16				
32-bit integer add, extended-precision add, subtract, extended-precision subtract	128		64	128		64				

IPC for different operations

<https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#arithmetic-instructions>



Little's law for GPU

- Concept of latency and concurrency apply to warp

$$\text{Warp Concurrency} = \text{Instruction Latency} * \text{Instruction Throughput} * \frac{1}{ILP}$$

Example

- A SM of NVIDIA V100 theoretical peak FP32 Add throughput is 64 IPC. Each FP32 Add instruction has 4 cycle latency. Assume each warp does one FP32 Add instruction at a time, how many active warps is needed in concurrent for achieving the theoretical peak throughput of SM?

$$\text{Warp Concurrency} = \text{Instruction Latency} * \text{InstructionThroughput} * \frac{1}{ILP} = 4 * \frac{64}{32} * 1 = 8 \text{ warps}$$

We need 8 warps executing at the same time to be able to achieve peak throughput

Example

- A SM of NVIDIA V100 theoretical peak FP64 Add throughput is 32 IPC. Each FP64 Add instruction has 4 cycle latency. Assume each warp does one FP64 Add instruction at a time, how many active warps is needed in concurrent for achieving the theoretical peak throughput of V100 GPU with 84 SMs?

$$\text{Warp Concurrency} = \text{Instruction Latency} * \text{Instruction Throughput} * \frac{1}{\text{ILP}} = 4 * \frac{32}{32} * 1 = 4 \text{ warps}$$

$$\text{GPU Concurrency} = \text{Warp Concurrency} * \text{\#of SM} = 4 * 84 = 336 \text{ warps}$$

We need 336 warps executing at the same time to be able to achieve peak throughput

Example

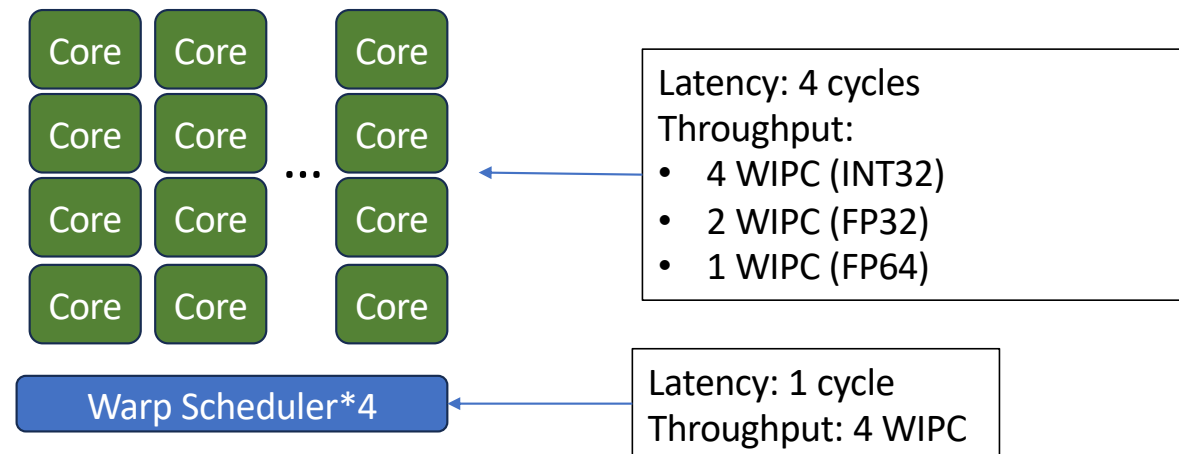
- A SM of NVIDIA V100 theoretical peak FP32 Add throughput is 64 IPC. Each FP32 Add instruction has 4 cycle latency. Assume each warp does two independent FP32 Add instructions, how many active warps is needed in concurrent for achieving the theoretical peak throughput of SM?

$$\text{Warp Concurrency} = \text{Instruction Latency} * \text{Instruction Throughput} * \frac{1}{ILP} = 4 * \frac{64}{32} * \frac{1}{2} = 4 \text{ warps}$$

We need 4 warps executing at the same time to be able to achieve peak throughput

Mismatching throughputs

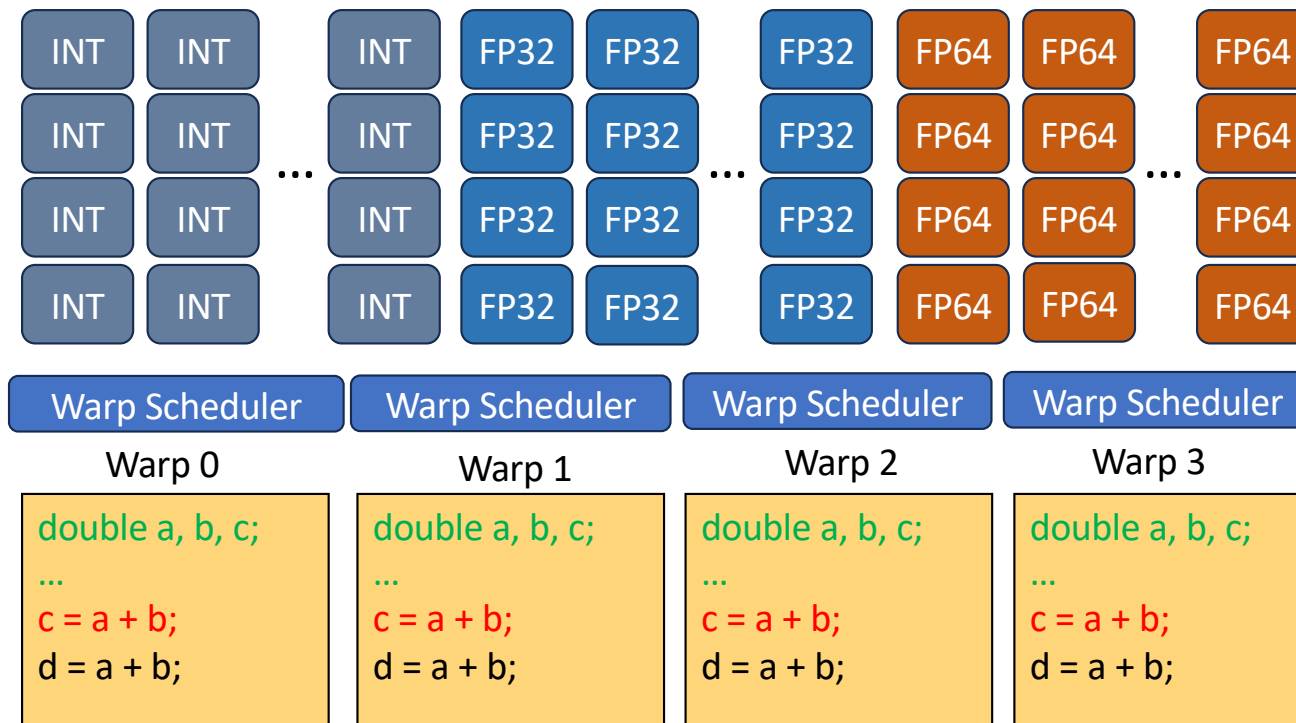
- CUDA cores and warp scheduler have different processing speed
- For example: GPU with sm_7.x (V100)



Warp schedulers may have nothing to issue because maximum concurrency has been reached

Which instruction/warp can be issued next?

Peak FP 64 throughput: 32 IPC = 1 WIPC
Maximum concurrency: 4/ILP



Completed : green
Executing : red
To execute: black

None of them can be issued



UNIVERSITY OF
OREGON

Computer Science

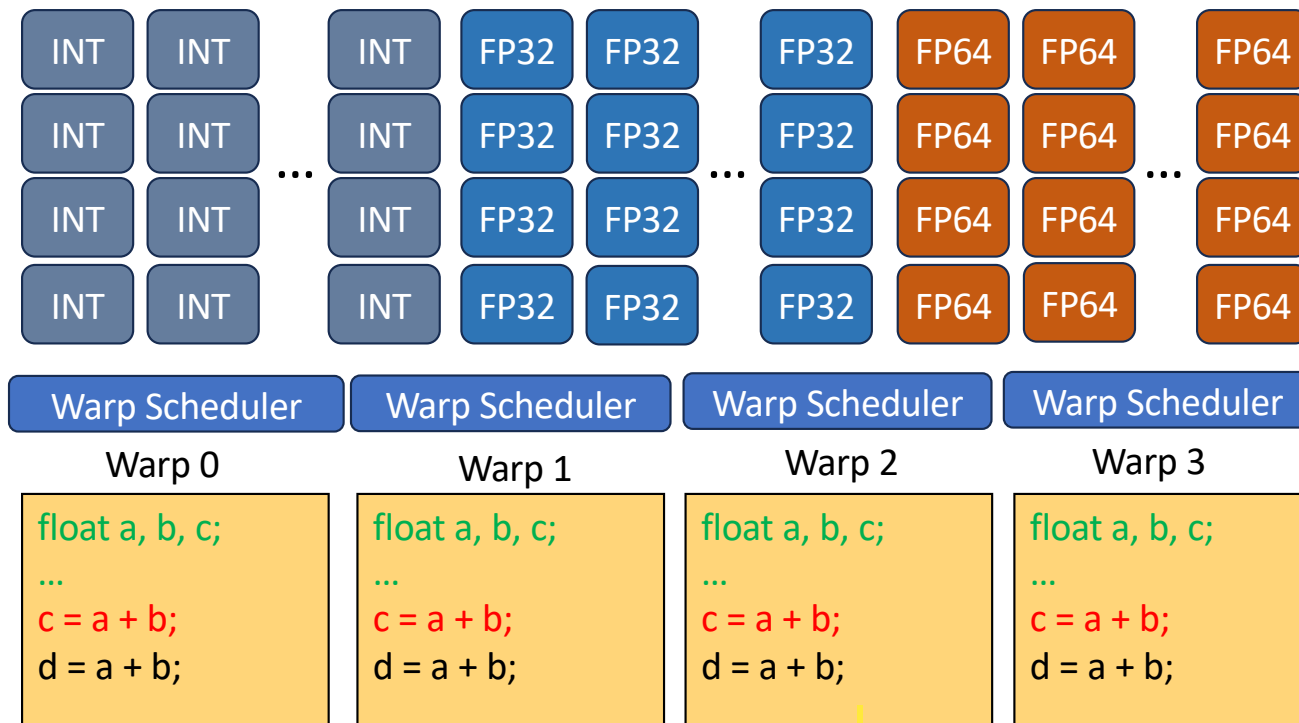
CS 510 GPU Programming - Spring 2026

63

Which instruction/warp can be issued next?

Peak FP 32 throughput: 64 IPC = 2 WIPC

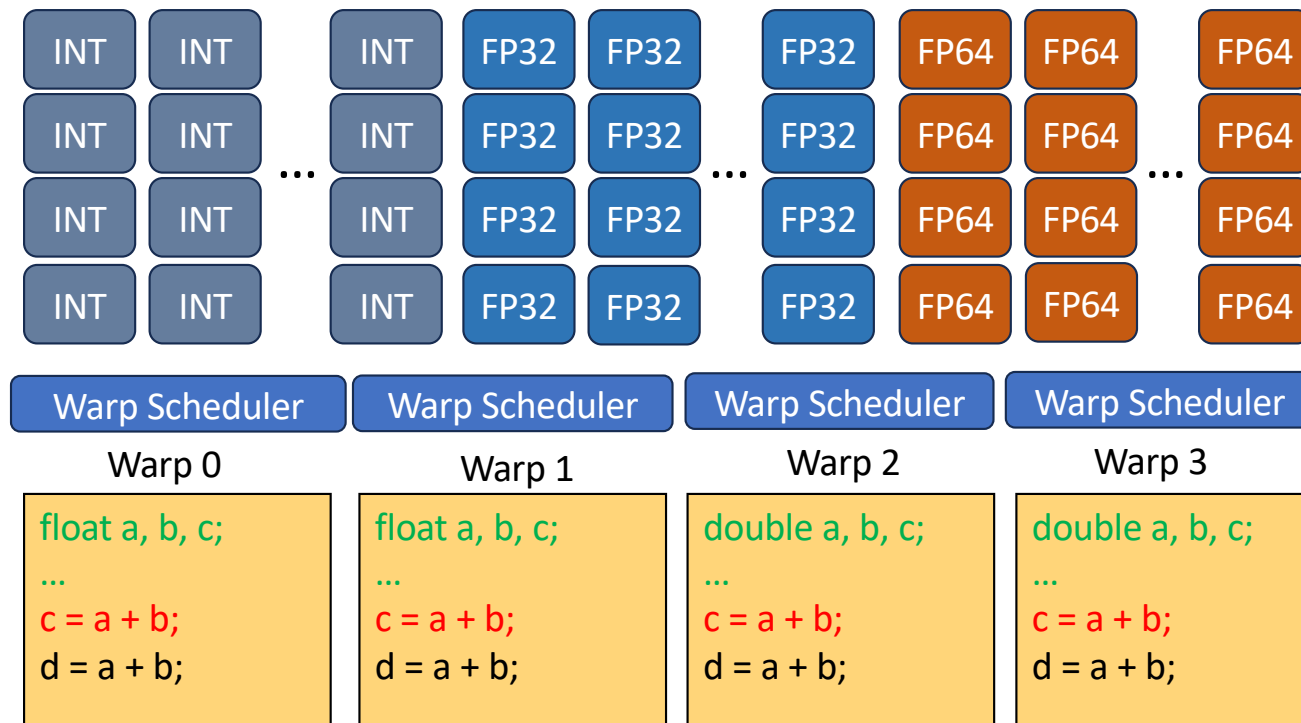
Maximum concurrency: 8/ILP



Which instruction/warp can be issued next?

Peak FP 32 throughput: 64 IPC = 2 WIPC
Maximum concurrency: 8/ILP

Peak FP 64 throughput: 32 IPC = 1 WIPC
Maximum concurrency: 4/ILP



Completed : green
Executing : red
To execute: black



UNIVERSITY OF
OREGON

Computer Science

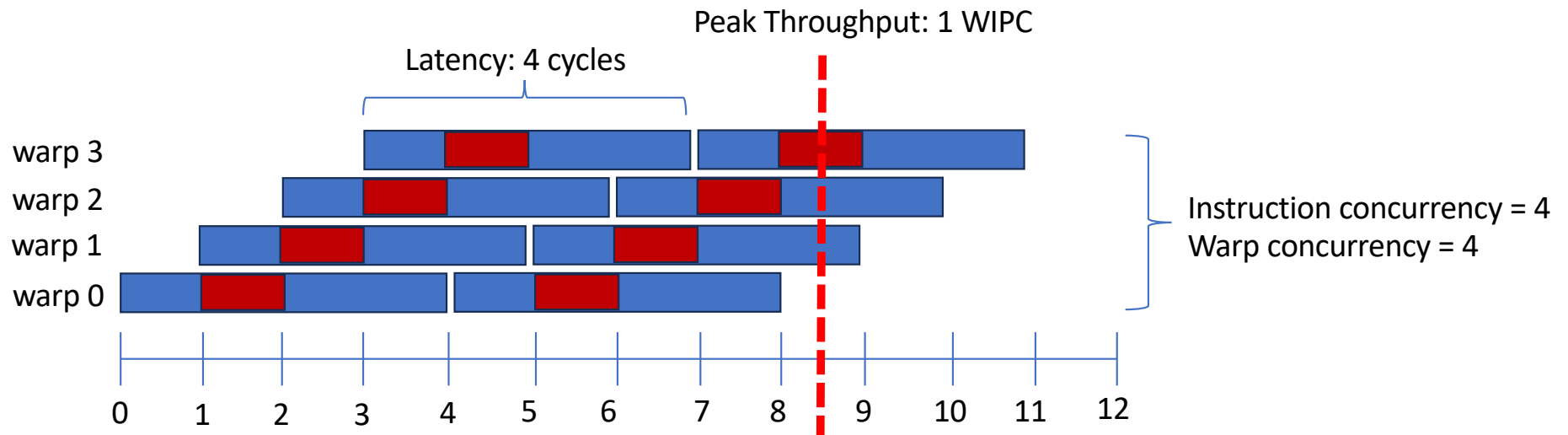
CS 610 GPU Programming – Winter 2026

65

Does warp concurrency always equals instruction concurrency?

- No. It depends.
- If a warp issues
 - **same-type** instruction every time
 - instructions have **back-to-back dependency**
- Then, warp concurrency == instruction concurrency

```
double a[4] // register array
double sum;
sum = sum + a[0];
sum = sum + a[1];
sum = sum + a[2];
sum = sum + a[3];
```

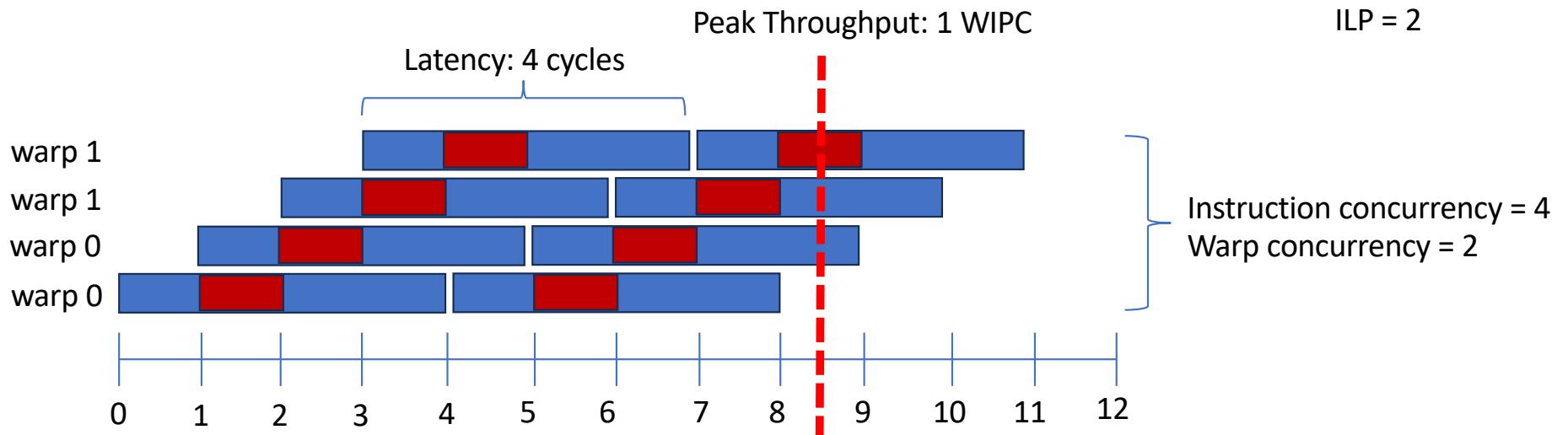


Does warp concurrency always equals instruction concurrency?

- No. It depends.
- If a warp issues
 - **same-type** instruction every time
 - instructions are **independent** (ILP>1)
- Then, warp concurrency < instruction concurrency

```
double a[5] // register array
a[0] = a[1] + a[0];
a[2] = a[1] + a[2];
a[3] = a[0] + a[3];
a[4] = a[2] + a[4];
```

ILP = 2

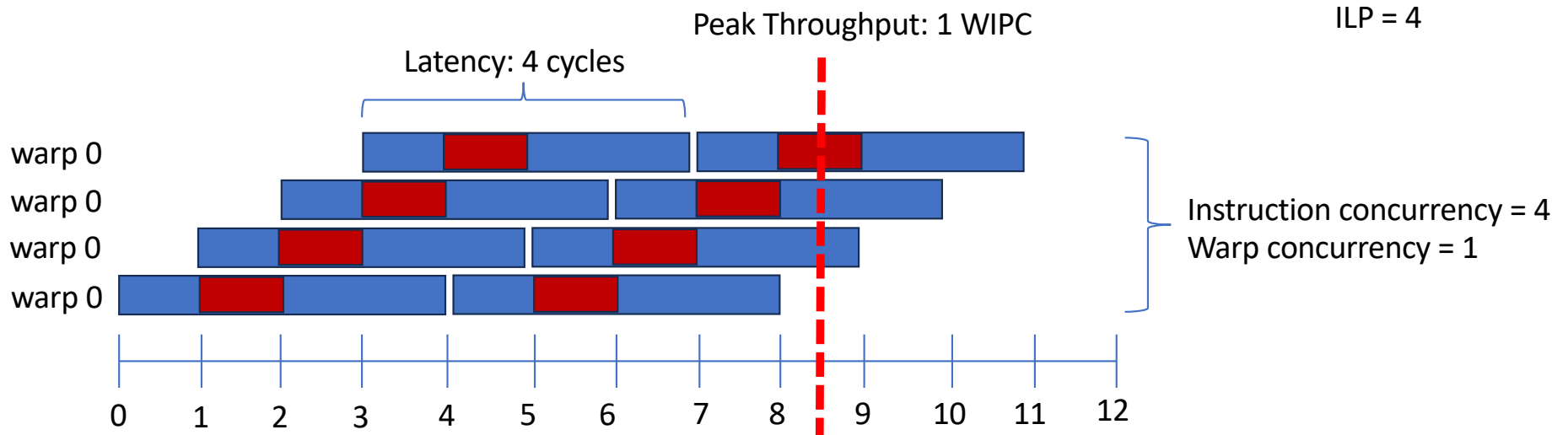


Does warp concurrency always equals instruction concurrency?

- No. It depends.
- If a warp issues
 - **same-type** instruction every time
 - instructions are **independent** (ILP>1)
- Then, warp concurrency < instruction concurrency

```
double a[5] // register array
a[0] = a[0] + a[0];
a[2] = a[2] + a[2];
a[3] = a[3] + a[3];
a[4] = a[4] + a[4];
```

ILP = 4



Does warp concurrency always equals instruction concurrency?

- No. It depends.
- If a warp issues
 - **mixed-type** instruction every time
 - instructions have **back-to-back dependency**
- Then, warp concurrency > instruction concurrency

```
double a[4] // register array
int b[4] // register array
double sum;
int i = 0;
i = i + b[0];
sum = sum + a[i];
i = i + b[1];
sum = sum + a[i];
i = i + b[2];
sum = sum + a[i];
i = i + b[3];
sum = sum + a[i];
```

Mixing INT with FP64

Instruction ratios:

- INT: 50%
- FP64: 50%

Mixed type instructions prolong the warp latency, so we need more warps to achieving the same instruction (computing) throughput

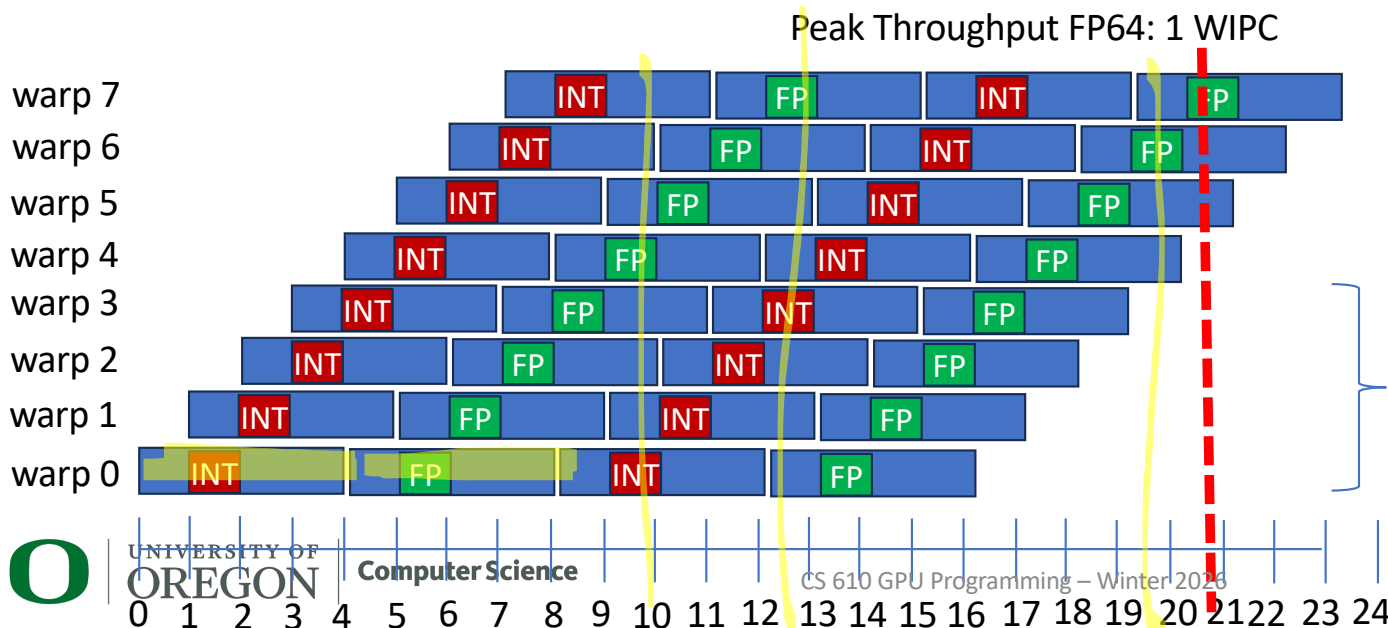
$$\text{Warp Concurrency} = \frac{\text{Instruction Latency} * \text{Instruction Throughput}}{\text{ILP} * \text{instruction ratio}}$$

Does warp concurrency always equals instruction concurrency?

- No. It depends.
- If a warp issues
 - **mixed-type** instruction every time
 - instructions have **back-to-back dependency**
- Then, warp concurrency > instruction concurrency

```
double a[4] // register array
int b[4] // register array
double sum;
int i = 0;
i = i + b[0];
sum = sum + a[i];
i = i + b[1];
sum = sum + a[i];
i = i + b[2];
sum = sum + a[i];
i = i + b[3];
sum = sum + a[i];
```

Mixing INT with FP64 (50%)



Instruction concurrency = 4
Warp concurrency = 8

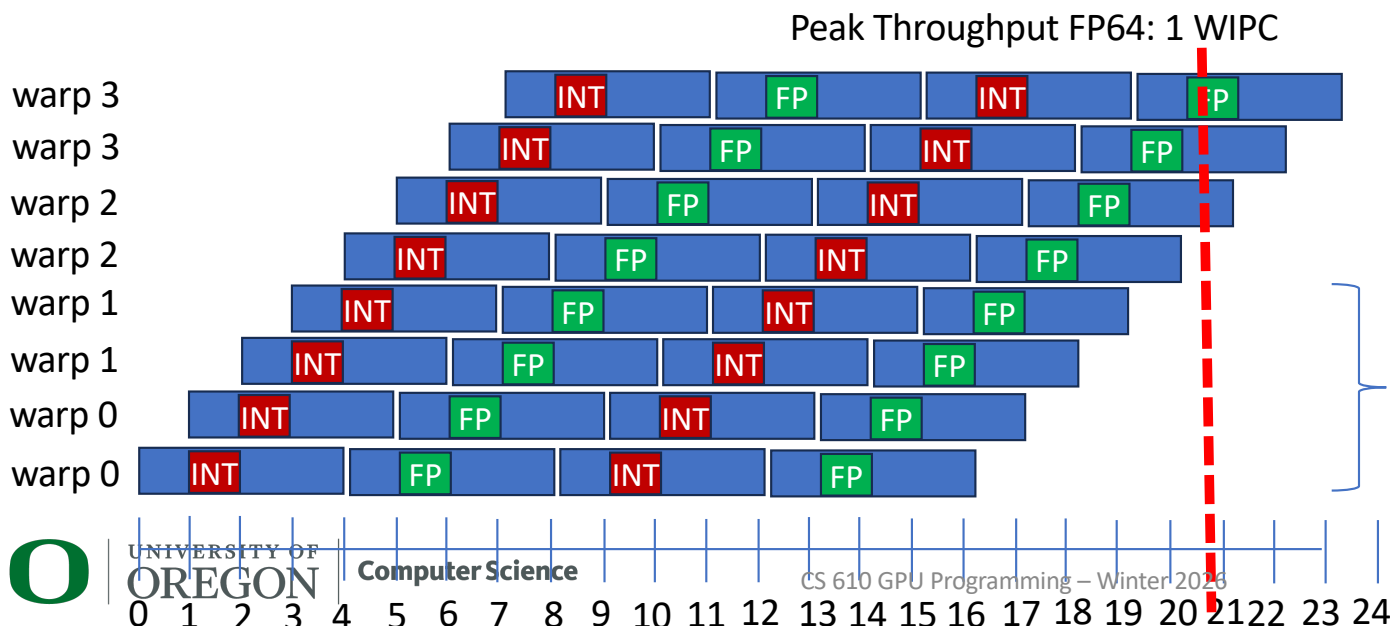
Warp concurrency * (1/50%) = 4

Does warp concurrency always equals instruction concurrency?

- No. It depends.
- If a warp issues
 - **mixed-type** instruction every time
 - instructions are **independent** (ILP>1)
- Then, warp concurrency $\neq$ instruction concurrency

```
double a[4] // register array
int b[4] // register array
double sum;
int i=0, j=0;
i = i + b[0];
j = j + b[1];
a[i] = a[i] * a[i];
a[j] = a[j] * a[j];
i = i + b[2];
j = j + b[3];
a[i] = a[i] * a[i];
a[j] = a[j] * a[j];
```

Mixing INT with FP64 (50%)
ILP=2



Instruction concurrency = 4
Warp concurrency = 4

Warp concurrency * (1/(50% * 2)) = 4

Does warp concurrency always equals instruction concurrency?

- It depends
 - Same-type + Dependent: warp == instruction
 - Same-type + Independent: warp < instruction
 - Mixed-type + Dependent: warp > instruction
 - Mixed-type + Independent: warp ? instruction (depends on instruction ratio and ILP)
- Improved Little's law

$$\text{Warp Concurrency} = \frac{\text{Instruction Latency} * \text{Instruction Throughput}}{\text{ILP} * \text{instruction ratio}}$$

What does it imply?

$$\text{Warp Concurrency} = \frac{\text{Instruction Latency} * \text{Instruction Throughput}}{\text{ILP} * \text{instruction ratio}}$$

$$\text{Instruction Throughput} = \frac{\text{Warp Concurrency} * \text{ILP} * \text{instruction ratio}}{\text{Instruction Latency}}$$

- To maximize computation throughput, we need to maximize instruction throughput
- We need to improve
 - Warp concurrency
 - Instruction level parallelism
 - Instruction ratio

How to improve warp concurrency?

- Improve total number of active warps in a SM

$$\textit{Active Warp} = \textit{Warp per TB} * \textit{TB per SM}$$

Which one has better warp concurrency?

Kernel1<<<10, 512>>>(...)

Kernel2<<<5, 512>>>(...)

Answer: we don't know

Because we don't know how many TB can reside in a SM

We need to study SM occupancy

Maximizing Occupancy



CS 610 GPU Programming, Winter 2026

What is occupancy

- Each SM can hold certain number of threads
 - All threads reside in SM potentially can contribute concurrency
 - Including executing threads and inactive threads

	Compute Capability													
Technical Specifications	5.0	5.2	5.3	6.0	6.1	6.2	7.0	7.2	7.5	8.0	8.6	8.7	8.9	9.0
Maximum number of resident warps per SM	64							32	64	48			64	
Maximum number of resident threads per SM	2048							1024	2048	1536			2048	

$$Occupancy = \frac{\text{Threads Resides in a SM}}{\text{Max Thread Allowed Per SM}}$$

Example

- A SM of H100 can hold 2048 threads. When Kernel_B runs on H100, each SM can hold 8 thread blocks. Each thread block has 8 warps. What is the occupancy of running Kernel_B on H100?

$$Occupancy = \frac{8 * 8 * 32}{2048} = 100\%$$

It means we used 100% of the parallelism that a GPU can provide

Example

- A SM of V100 can hold 2048 threads. When Kernel_A runs on V100, each SM can hold 2 thread blocks. Each thread block has 16 warps. What is the occupancy of running Kernel_A on V100?

$$Occupancy = \frac{2 * 16 * 32}{2048} = 50\%$$

It means we only used 50% of the parallelism that a GPU can provide

Occupancy vs. Concurrency

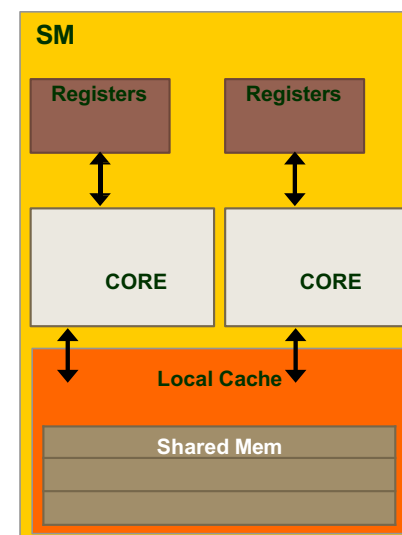
- Occupancy indicate the number of threads (warps) that may potentially issue instruction.
- Concurrency indicate the number of threads (warps) that are actually issuing instruction.
- Higher occupancy means the there is higher chance for warp schedulers to find a warp that is ready to execute

Does low occupancy mean bad performance?

- Not necessarily
- We can improve ILP and instruction ratio to improve concurrency
- But it is always beneficial if we can improve occupancy

What affects occupancy?

- What controls how many threads or thread blocks can reside in a SM?
- HW resource per SM is limited, so we need to consider:
 - Register usage per thread
 - Shared memory usage per thread block
 - Thread block size



SM resource limitations

	Compute Capability													
Technical Specifications	5.0	5.2	5.3	6.0	6.1	6.2	7.0	7.2	7.5	8.0	8.6	8.7	8.9	9.0
Number of 32-bit registers per SM	64 K													
Maximum number of 32-bit registers per thread block	64 K		32 K	64 K		32 K	64 K							
Maximum number of 32-bit registers per thread	255													
Maximum amount of shared memory per SM	64 KB	96 KB	64 KB		96 KB	64 KB	96 KB		64 KB	164 KB	100 KB	164 KB	100 KB	228 KB
Maximum amount of shared memory per thread block ³²	48 KB						96 KB	96 KB	64 KB	163 KB	99 KB	163 KB	99 KB	227 KB
Maximum number of resident blocks per SM	32								16	32	16		24	32

Estimating occupancy without running kernel

$$TB_{reg} = \left\lfloor \frac{\frac{\text{Total Reg. per SM}}{\text{Reg. per Thread}}}{TB \text{ Size}} \right\rfloor$$

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor$$

$$Occupancy = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}}$$

It is useful to estimate kernel occupancy so we can adjust our kernel to improve occupancy



Find out the reason of low occupancy

Could be one or several of following reasons:

1. Register usage per thread
2. Shared memory usage per thread block
3. Thread block size

Example

- A SM of V100 can hold 2048 threads. Kernel_C's thread block contains 1024 threads. Each thread uses 40 32-bit registers. Each thread block uses 60KB shared memory. What would be the occupancy?

Occupancy is limited by both register usage and shared memory usage

$$TB_{reg} = \left\lfloor \frac{\text{Total Reg. per SM}}{\text{Reg. per Thread}} \right\rfloor = \left\lfloor \frac{65536}{40} \right\rfloor = 1$$

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor = \left\lfloor \frac{90}{60} \right\rfloor = 1$$

$$\text{Occupancy} = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}} = \frac{\min(1, 1, 32) * 1024}{2048} = 50\%$$



Example

- A SM of V100 can hold 2048 threads. Kernel_C's thread block contains 1024 threads. Each thread uses 10 32-bit registers. Each thread block uses 60KB shared memory. What would be the occupancy?

Occupancy is limited by shared memory usage

$$TB_{reg} = \left\lfloor \frac{\frac{\text{Total Reg. per SM}}{\text{Reg. per Thread}}}{TB \text{ Size}} \right\rfloor = \left\lfloor \frac{\frac{65536}{40}}{1024} \right\rfloor = 6$$

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor = \left\lfloor \frac{90}{60} \right\rfloor = 1$$

$$\text{Occupancy} = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}} = \frac{\min(6, 1, 32) * 1024}{2048} = 50\%$$



Example

- A SM of V100 can hold 2048 threads. Kernel_C's thread block contains 1024 threads. Each thread uses 40 32-bit registers. Each thread block uses 10KB shared memory. What would be the occupancy?

Occupancy is limited by register usage

$$TB_{reg} = \left\lfloor \frac{\frac{\text{Total Reg. per SM}}{\text{Reg. per Thread}}}{TB \text{ Size}} \right\rfloor = \left\lfloor \frac{\frac{65536}{40}}{1024} \right\rfloor = 1$$

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor = \left\lfloor \frac{90}{10} \right\rfloor = 9$$

$$\text{Occupancy} = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}} = \frac{\min(1, 9, 32) * 1024}{2048} = 50\%$$



How to estimate the resource usage of your kernel?

- Compile with

```
nvcc --ptxas-options=-v test.cu
```

NVCC will report per kernel profile

```

__global__ void kernel_1(int n, int * data){

    int A[4];
    A[0] = data[0];
    A[1] = data[1];
    A[2] = data[2];
    A[3] = data[3];
    A[0] = A[0] + A[1];
    A[1] = A[1] + A[2];
    A[2] = A[2] + A[3];
    A[3] = A[3] + A[0];

    data[0] = A[0] * A[1];
    data[1] = A[1] * A[2];
    data[2] = A[2] * A[3];
    data[3] = A[3] * A[0];
}

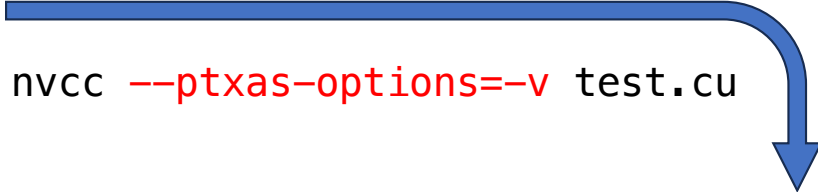
__global__ void kernel_2(int n, int * data){

    __shared__ int sm[4];
    int A[4];
    A[0] = data[0];
    A[1] = data[1];
    A[2] = data[2];
    A[3] = data[3];
    sm[0] = A[0] + A[1];
    sm[1] = A[1] + A[2];
    sm[2] = A[2] + A[3];
    sm[3] = A[3] + A[0];

    data[0] = sm[0] * sm[1];
    data[1] = sm[1] * sm[2];
    data[2] = sm[2] * sm[3];
    data[3] = sm[3] * sm[0];
}

```

nvcc --ptxas-options=-v test.cu



```

ptxas info      : Compiling entry function '_Z8kernel_2iPi' for 'sm_52'
ptxas info      : Function properties for _Z8kernel_2iPi
                  0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 16 registers, 16 bytes smem, 336 bytes cmem[0]
ptxas info      : Compiling entry function '_Z8kernel_1iPi' for 'sm_52'
ptxas info      : Function properties for _Z8kernel_1iPi
                  0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 16 registers, 336 bytes cmem[0]

```

Per kernel profile

- **cmem** stands for constant memory
- **gmem** stands for global memory
- **smem** stands for shared memory
- **lmem** stands for local memory
- **stack frame**: part of local memory
- **spill loads** and **store**: the amount of register spill
- If not shown it means zero usage

Getting the register and shared memory usage

```
ptxas info      : Compiling entry function '_Z8kernel_2iPi' for 'sm_52'
ptxas info      : Function properties for _Z8kernel_2iPi
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 16 registers, 16 bytes smem, 336 bytes cmem[0]
ptxas info      : Compiling entry function '_Z8kernel_1iPi' for 'sm_52'
ptxas info      : Function properties for _Z8kernel_1iPi
    0 bytes stack frame, 0 bytes spill stores, 0 bytes spill loads
ptxas info      : Used 16 registers 336 bytes cmem[0]
```

Register Usage

- Reported usage may be different than what you expected
 - Compiler may allocate more registers for saving intermediate results
 - Compiler may also reuse some registers
- Compiler makes the ultimate decisions on register usage
- The primary goal of compiler
 - Minimize register spilling (use more registers for thread private memory)
 - Minimize # of instructions (use more registers to keep intermediate results and improve ILP)

Minimize register usage

- Remove unnecessary calculations
- Replacing constant terms with marco or template variables
- Split one kernel into multiple kernels (cost extra launch overhead)

Shared Memory Usage

- Total usage = static allocated + dynamically allocated

```
__global__ kernel() {  
    __shared__ int A[100];  
    extern __shared__ int B[];  
}
```

kernel<<<1, N, 200>>>()

Total usage = (100+200) * sizeof(int)

Configure shared memory space

- Shared memory and L1 cache use the same physical memory space
- We can configure how much of that space is reserved for dynamic shared memory

```
cudaFuncSetAttribute(my_kernel,  
cudaFuncAttributeMaxDynamicSharedMemorySize,  
64*1024);
```



SM resource limitations

	Compute Capability													
Technical Specifications	5.0	5.2	5.3	6.0	6.1	6.2	7.0	7.2	7.5	8.0	8.6	8.7	8.9	9.0
Number of 32-bit registers per SM	64 K													
Maximum number of 32-bit registers per thread block	64 K	32 K	64 K	32 K	64 K									
Maximum number of 32-bit registers per thread	255													
Maximum amount of shared memory per SM	64 KB	96 KB	64 KB	96 KB	64 KB	96 KB	64 KB	164 KB	100 KB	164 KB	100 KB	228 KB		
Maximum amount of shared memory per thread block ³²	48 KB						96 KB	96 KB	64 KB	163 KB	99 KB	163 KB	99 KB	227 KB
Maximum number of resident blocks per SM	32								16	32	16	24	32	

Example

- A SM of V100 can hold 2048 threads. Kernel_C's thread block contains 1024 threads. Each thread uses 10 32-bit registers. Each thread block uses 40KB shared memory. What would be the occupancy?

$$TB_{reg} = \left\lfloor \frac{\text{Total Reg. per SM}}{\text{Reg. per Thread}} \right\rfloor = \left\lfloor \frac{65536}{40} \right\rfloor = 6$$

```
cudaFuncSetAttribute(my_kernel,  
cudaFuncAttributeMaxDynamicSharedMemorySize,  
60*1024);
```

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor = \left\lfloor \frac{60}{40} \right\rfloor = 1$$

$$\text{Occupancy} = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}} = \frac{\min(6, 1, 32) * 1024}{2048} = 50\%$$



Example

- A SM of V100 can hold 2048 threads. Kernel_C's thread block contains 1024 threads. Each thread uses 10 32-bit registers. Each thread block uses 40KB shared memory. What would be the occupancy?

$$TB_{reg} = \left\lfloor \frac{\text{Total Reg. per SM}}{\text{Reg. per Thread}} \right\rfloor = \left\lfloor \frac{65536}{40} \right\rfloor = 6$$

```
cudaFuncSetAttribute(my_kernel,  
cudaFuncAttributeMaxDynamicSharedMemorySize,  
90*1024);
```

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor = \left\lfloor \frac{90}{40} \right\rfloor = 2$$

$$\text{Occupancy} = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}} = \frac{\min(6, 2, 32) * 1024}{2048} = 100\%$$



Choosing thread block size

- ❑ Changing the thread block size will change occupancy
 - ❑ If thread block size is too small
 - ❑ There is a fixed limit on the number of active thread blocks per SM (16 in Kepler, 32 in
 - Maxwell)
 - ❑ If thread block is too big
 - ❑ Not enough resources for another block
 - ❑ Block is stalled until enough resource is available

SM resource limitations

	Compute Capability													
Technical Specifications	5.0	5.2	5.3	6.0	6.1	6.2	7.0	7.2	7.5	8.0	8.6	8.7	8.9	9.0
Number of 32-bit registers per SM	64 K													
Maximum number of 32-bit registers per thread block	64 K		32 K	64 K		32 K	64 K							
Maximum number of 32-bit registers per thread	255													
Maximum amount of shared memory per SM	64 KB	96 KB	64 KB		96 KB	64 KB	96 KB		64 KB	164 KB	100 KB	164 KB	100 KB	228 KB
Maximum amount of shared memory per thread block ³²	48 KB						96 KB	96 KB	64 KB	163 KB	99 KB	163 KB	99 KB	227 KB
Maximum number of resident blocks per SM	32								16	32	16		24	32

Example

- A SM of V100 can hold 2048 threads. Kernel_C's thread block contains 32 threads. Each thread uses 10 32-bit registers. Each thread block uses 2KB shared memory. What would be the occupancy?

$$TB_{reg} = \left\lfloor \frac{\text{Total Reg. per SM}}{\frac{\text{Reg. per Thread}}{TB \text{ Size}}} \right\rfloor = \left\lfloor \frac{65536}{\frac{10}{32}} \right\rfloor = 204$$

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor = \left\lfloor \frac{90}{2} \right\rfloor = 45$$

$$\text{Occupancy} = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}} = \frac{\min(204, 45, 32) * 32}{2048} = 50\%$$

Occupancy is limited by thread block size (too small). Each SM of V100 can only hold maximum 32 TBs regardless of resource usage



Example

- A SM of V100 can hold 2048 threads. Kernel_C's thread block contains 1024 threads. Each thread uses 40 32-bit registers and 60 bytes shared memory. What would be the occupancy?

$$TB_{reg} = \left\lfloor \frac{\frac{\text{Total Reg. per SM}}{\text{Reg. per Thread}}}{TB \text{ Size}} \right\rfloor = \left\lfloor \frac{\frac{65536}{40}}{1024} \right\rfloor = 1$$

Occupancy is limited by both register usage and shared memory usage

What if we decrease block size?

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor = \left\lfloor \frac{90 * 1024}{60 * 1024} \right\rfloor = 1$$

$$\text{Occupancy} = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}} = \frac{\min(1, 1, 32) * 1024}{2048} = 50\%$$



Example

- A SM of V100 can hold 2048 threads. Kernel_C's thread block contains 512 threads. Each thread uses 40 32-bit registers and 60 bytes shared memory. What would be the occupancy?

Occupancy is limited by both register usage and shared memory usage

$$TB_{reg} = \left\lfloor \frac{\text{Total Reg. per SM}}{\text{Reg. per Thread}} \right\rfloor = \left\lfloor \frac{65536}{40} \right\rfloor = 3$$

What if we decrease block size?
Smaller TB size can help us fit more TB per SM

$$TB_{shared} = \left\lfloor \frac{\text{Total Shared Mem. per SM}}{\text{Shared Mem. per TB}} \right\rfloor = \left\lfloor \frac{90 * 1024}{60 * 512} \right\rfloor = 3$$

$$\text{Occupancy} = \frac{\min(TB_{reg}, TB_{shared}, TB_{\max \text{ per SM}}) * TB \text{ Size}}{\text{Max Thread Allowed Per SM}} = \frac{\min(3, 3, 32) * 512}{2048} = 75\%$$

